M.D.S

CREATED BY MAHDI CHERIF

LE GUIDE COMPLET

RÉUSSIR SON PFE

Le guide complet : de la recherche de stage à la soutenance.





1 RECHERCHE — 2 ENTRETIEN — 3 PROJET — 4 RAPPORT — 5 SOUTENANCE

MAHDI CHERIF

M.D.S — Created by Mahdi Cherif

ÉDITION 2026

M.D.S
CREATED BY MAHDI CHERIF

RÉUSSIR SON PFE

Le guide complet : de la recherche de stage à la soutenance.

Auteur et créateur : Mahdi Cherif
Signature : M.D.S — Created by Mahdi Cherif
Public visé : étudiants en informatique et en génie logiciel, principalement en Tunisie, et plus largement dans l'espace francophone.

© Tous droits réservés — M.D.S — Mahdi Cherif

Aucune partie de ce guide ne peut être reproduite, diffusée ou vendue sans l'accord écrit de son auteur. Les modèles (CV, lettres, messages, checklists) sont fournis pour un usage personnel : adaptez-les, ne les copiez pas tels quels.

À VÉRIFIER AUPRÈS DE VOTRE ÉTABLISSEMENT

Nature de ce guide

Ce document est un guide d'accompagnement. Il ne remplace ni le règlement de votre établissement, ni les consignes de vos encadrants. Les exigences de rapport, de soutenance et de convention de stage varient d'un établissement à l'autre : le guide distingue toujours **conseils personnels** et **exigences officielles**, et ne contient ni statistiques ni références inventées.

« M.D.S » est ici une identité typographique créée à partir des initiales de la marque ; il ne s'agit pas d'un logo officiel déposé. Les noms de produits et de technologies cités appartiennent à leurs propriétaires respectifs. Les fonctionnalités des outils évoluent vite : vérifiez toujours leur documentation officielle.

Avant-propos

Un PFE n'est pas seulement un projet technique : c'est un parcours qui mêle recherche d'opportunité, communication, gestion de projet, ingénierie logicielle, écriture académique et prise de parole.

Beaucoup d'étudiants abordent ce parcours morceau par morceau, souvent dans l'urgence. Ce guide propose l'inverse : une vue d'ensemble, dans l'ordre chronologique, avec des méthodes, des exemples et des outils réutilisables. Chaque partie peut se lire seule, mais la progression complète vous évite les mauvaises surprises.



Figure 1 — Les cinq moments d'un PFE et les parties du guide qui les couvrent.

Comment utiliser ce guide

- **Lisez d'abord la partie qui correspond à votre étape actuelle**, puis parcourez les autres pour anticiper la suite.
- **Remplissez les outils** de la Boîte à outils (fin du guide) : suivi des candidatures, cahier des charges, planning, checklists.
- **Adaptez les exemples** : les textes entre crochets [comme ceci] sont des espaces à personnaliser avec vos informations réelles.
- **Gardez une trace écrite** : notes de réunion, décisions, captures d'écran — elles nourriront votre rapport.

Les encadrés du guide

Conseil — une recommandation d'expérience, à adapter à votre contexte.

Attention — un piège fréquent ou un risque à éviter.

À vérifier — une exigence qui dépend de votre établissement : ne la supposez pas.

Définition — un terme technique expliqué avant d'être utilisé.

À savoir — un complément utile pour comprendre.

À retenir — l'essentiel, à mémoriser.

Rappel : selon les établissements, le PFE peut s'appeler « projet de fin d'études », « stage d'ingénieur » ou « mémoire de fin d'études ». Dans ce guide, **encadrant académique** désigne l'enseignant qui suit votre travail et **encadrant professionnel** la personne qui vous encadre en entreprise.

Sommaire

Partie 1 — Préparer son entrée dans le monde professionnel 7

1.1 Comprendre ce que recherche une entreprise 8
1.2 Identifier les entreprises qui proposent des PFE 9
1.3 Mener une recherche efficace 10
1.4 Utiliser LinkedIn comme un professionnel 11
1.5 Contacter les recruteurs 12
1.6 Envoyer une candidature par e-mail 13
1.7 Relancer et organiser son suivi 13

Partie 2 — Construire un CV et une lettre de motivation 15

2.1 Les sections d'un CV informatique 16
2.2 Présenter ses compétences techniques 16
2.3 Valoriser ses projets académiques 17
2.4 Présenter ses certifications 18
2.5 Choisir un design professionnel 18
2.6 Les erreurs qui éliminent une candidature 19
2.7 Exemple de CV pour un étudiant en informatique 20
2.8 Rédiger une lettre de motivation personnalisée 21
2.9 E-mail de candidature et message LinkedIn 22

Partie 3 — Réussir son entretien et intégrer l'entreprise 23

3.1 Préparer son entretien 24
3.2 Se présenter en une à deux minutes 25
3.3 Les questions RH classiques 25
3.4 Les questions techniques possibles 26
3.5 Parler de ses projets universitaires 28
3.6 Quand on ne connaît pas une technologie 28
3.7 Simulation d'entretien commentée 29
3.8 Les comportements professionnels à adopter 30
3.9 Communiquer avec son encadrant et comprendre les attentes 31

Partie 4 — Organiser et planifier son projet PFE 32

4.1 Comprendre le besoin et formuler la problématique 33
4.2 Définir les objectifs et le périmètre 34
4.3 Préparer un cahier des charges 34
4.4 Rédiger des User Stories 35
4.5 Utiliser Scrum et Kanban 36
4.6 Répartir les tâches 37
4.7 Établir un planning sur 4 à 6 mois 38

4.8 Gérer les réunions et les comptes rendus . . . 39
4.9 Travailler avec un encadrant professionnel . . . 40

Partie 5 — Conception et développement technique . . . 41

5.1 Choisir une stack adaptée au projet . . . 42
5.2 Architecture d'une application et modèle client-serveur . . . 43
5.3 Front-end et back-end . . . 44
5.4 Bases de données relationnelles et non relationnelles . . . 45
5.5 Modélisation UML . . . 46
5.6 Concevoir une API REST . . . 49
5.7 Authentification et autorisation . . . 50
5.8 Sécurité informatique . . . 51
5.9 Git et GitHub . . . 53
5.10 Tests logiciels . . . 54
5.11 Déploiement et documentation . . . 55

Partie 6 — Rédiger un rapport PFE professionnel . . . 57

6.1 Avant de commencer : gabarit et méthode . . . 58
6.2 La structure complète du rapport . . . 58
6.3 Le contenu attendu dans chaque chapitre . . . 59
6.4 Exemples de rédaction académique . . . 60
6.5 Mise en page, figures, tableaux . . . 61
6.6 Citer ses sources . . . 61
6.7 Word, WPS Office ou LibreOffice : réussir la mise en forme . . . 62

Partie 7 — Préparer une soutenance exceptionnelle . . . 63

7.1 Comprendre le format de la soutenance . . . 64
7.2 Sélectionner l'essentiel . . . 64
7.3 Structure type de 12 à 15 diapositives . . . 64
7.4 Présenter problématique, solution et architecture . . . 66
7.5 Réussir sa démonstration . . . 66
7.6 Gérer le temps et le stress . . . 67
7.7 Répondre aux questions du jury . . . 68
7.8 30 questions possibles et pistes de réponses . . . 68

Partie 8 — Les erreurs à éviter et les conseils pour se démarquer . . . 71

8.1 Les huit erreurs les plus coûteuses . . . 72
8.2 Comment se démarquer pendant son stage PFE ? . . . 72
8.3 Après la soutenance . . . 73

Chapitre spécial — L'intelligence artificielle au service du PFE . . . 74

A.1 Panorama des outils . . . 75
A.2 Sept usages concrets . . . 75

A.3 Douze prompts prêts à copier-coller 76
A.4 Limites, risques et bonnes pratiques 77

Boîte à outils — Boîte à outils : dix ressources prêtes à l'emploi 79

Outil 1 — Checklist de préparation du CV 80
Outil 2 — Checklist de candidature 81
Outil 3 — Tableau de suivi des entreprises contactées 81
Outil 4 — Checklist de préparation à l'entretien 82
Outil 5 — Modèle de cahier des charges 83
Outil 6 — Modèle de planning PFE 84
Outil 7 — Exemple de tableau Kanban 84
Outil 8 — Checklist de rédaction du rapport 85
Outil 9 — Checklist de préparation de la soutenance 86
Outil 10 — Planning des dernières semaines avant la soutenance 87

M.D.S

PARTIE 1

Préparer son entrée dans le monde professionnel

1

Identifier les bonnes entreprises, chercher efficacement, soigner sa présence en ligne, contacter les recruteurs et organiser son suivi.

AU PROGRAMME

1.1 Comprendre ce que recherche une entreprise

1.2 Identifier les entreprises qui proposent des PFE

1.3 Mener une recherche efficace

1.4 Utiliser LinkedIn comme un professionnel

1.5 Contacter les recruteurs

1.6 Envoyer une candidature par e-mail

1.7 Relancer et organiser son suivi

OUTILS ASSOCIÉS DE LA BOÎTE À OUTILS

Outil 3 — Tableau de suivi des entreprises contactées · Outil 2 — Checklist de candidature

1.1 Comprendre ce que recherche une entreprise

Avant de postuler, il faut comprendre pourquoi une entreprise accueille un stagiaire de PFE et ce qu'elle en attend.

Aa

DÉFINITION

PFE (Projet de Fin d'Études)

Projet réalisé en fin de cursus, le plus souvent au sein d'une entreprise, sur une durée de plusieurs mois (souvent entre 4 et 6 mois, selon l'établissement). Il donne lieu à un rapport écrit et à une soutenance devant un jury.

Pour l'entreprise, un stagiaire PFE représente à la fois une **ressource** qui traite un besoin réel, un **investissement en temps** (l'encadrant consacre des heures à votre formation) et, souvent, un **vivier de recrutement**. Cela explique ce que les recruteurs observent :

Ce que regarde le recruteur	Comment le démontrer
Des bases techniques solides	Projets académiques et personnels bien décrits, dépôt GitHub propre, réponses précises en entretien.
La capacité à apprendre vite	Exemples concrets de technologies apprises seul, de documentation lue, de problèmes résolus.
L'autonomie et la rigueur	Projets menés jusqu'au bout, code organisé, travail documenté, respect des délais.
La communication	Messages clairs, CV lisible, aisance pour expliquer un problème technique simplement.
La motivation pour *leur* sujet	Une candidature personnalisée qui montre que vous avez compris l'activité de l'entreprise.

Les types d'entreprises et leurs spécificités

Toutes les entreprises ne proposent pas le même type d'expérience. Aucun type n'est « meilleur » : l'important est qu'il corresponde à vos objectifs.

Type d'entreprise	Ce que vous pouvez y trouver	Points d'attention
Société de services (ESN)	Projets variés, méthodes industrialisées, contact avec plusieurs clients.	Vérifier que le sujet est bien un projet de fond et non une simple mission de renfort.
Start-up	Responsabilités rapides, stack moderne, forte autonomie.	Encadrement parfois limité ; périmètre qui peut changer vite.
Éditeur de logiciels	Produit à long terme, pratiques d'ingénierie établies.	Sujets parfois plus ciblés sur une brique du produit.
Banque, assurance, télécom, grande entreprise	Environnement structuré, volumes de données importants, exigences de sécurité.	Procédures d'accès parfois longues ; règles de confidentialité strictes.
PME ou entreprise industrielle	Besoin métier concret (gestion, production, logistique), grande visibilité de votre travail.	Peu de pairs techniques à proximité ; clarifier l'encadrement.

Type d'entreprise	Ce que vous pouvez y trouver	Points d'attention
Centre de recherche, laboratoire	Approfondissement scientifique, prototypes, publications possibles.	Sujet plus exploratoire ; livrables parfois moins « produit ».

CONSEIL

Définissez votre cap avant de chercher

Notez trois critères de choix : le **domaine** (web, mobile, data, cloud, sécurité…), l'**environnement de travail** (structure, taille d'équipe) et ce que vous voulez **apprendre**. Ils vous éviteront de postuler au hasard.

1.2 Identifier les entreprises qui proposent des PFE

Les offres de PFE ne sont pas toutes publiées. Une partie se joue dans les réseaux, les rencontres et les candidatures spontanées. Combinez donc plusieurs canaux.

Canal	Comment l'exploiter
LinkedIn	Offres d'emploi et de stage, recherche de publications « PFE », contacts directs avec recruteurs et responsables techniques.
Plateformes d'emploi locales	Sites d'annonces tunisiens (par exemple Keejob ou Tanitjobs) : créez des alertes sur les mots-clés « stage PFE », « développeur », « génie logiciel ».
Sites carrières des entreprises	Rubriques « Carrières » ou « Stages » des entreprises que vous ciblez : elles publient parfois avant les plateformes.
Votre établissement	Service des stages, forums entreprises, journées carrières, banque de sujets proposés par les enseignants.
Votre réseau	Anciens étudiants de votre filière, anciens stagiaires, enseignants, famille, amis : beaucoup d'offres se transmettent ainsi.
Événements technologiques	Hackathons, meetups, conférences, communautés de développeurs : l'occasion de rencontrer des équipes techniques.
Candidature spontanée	Pertinente quand l'entreprise ne publie rien mais travaille sur des technologies qui vous intéressent.

Constituer sa liste cible

Plutôt que d'envoyer des dizaines de messages identiques, construisez une **liste cible** structurée. Un tableur suffit (voir l'Outil 3 de la Boîte à outils).

1. Listez les entreprises par catégories : celles qui vous attirent fortement, celles qui sont réalistes, celles qui sont des options de sécurité.
2. Pour chacune, notez le site web, l'activité, la stack technique connue et un contact potentiel.
3. Repérez les entreprises qui ont déjà accueilli des stagiaires de votre établissement.
4. Classez par priorité et lancez-vous par vagues de quelques candidatures à la fois.

ATTENTION

Évitez les offres douteuses

Méfiez-vous d'une offre qui demande de l'argent, des documents personnels sensibles dès le premier contact, ou qui reste vague sur l'entreprise et le sujet.

Vérifiez l'existence de l'entreprise (site officiel, présence professionnelle cohérente) avant d'envoyer vos données.

1.3 Mener une recherche efficace

Une recherche efficace est régulière et méthodique. Quelques réglages vous font gagner un temps précieux.

Les bons mots-clés

Combinez le **type de stage**, le **domaine** et la **technologie**. Voici des exemples de requêtes à adapter (et à tester en français comme en anglais) :

EXEMPLES DE REQUÊTES

```
"stage PFE" développeur web
"PFE" "génie logiciel" Tunisie
"internship" "full stack" Tunisia
"PFE" Spring Boot OR React OR Angular
"stage de fin d'études" "ingénieur logiciel"
"PFE" DevOps OR cloud OR "data engineer"
```

Une routine hebdomadaire

Quand	Action	Durée indicative
Début de semaine	Consulter les alertes, trier les nouvelles offres, mettre à jour le suivi.	20 min
2 à 3 fois par semaine	Envoyer des candidatures personnalisées (pas plus que vous ne pouvez soigner).	1 h
En fin de semaine	Relancer les contacts en attente, remercier ceux qui ont répondu.	30 min
Chaque semaine	Enrichir son profil : un projet, une compétence, une publication.	1 h

Ces durées sont des repères personnels, pas des normes : ajustez-les à votre emploi du temps.

Lire une offre intelligemment

Une offre de PFE contient plus d'informations qu'il n'y paraît. Repérez : le **sujet** (est-il précis ou vague ?), les **technologies** demandées, la **durée**, la présence d'un **encadrant technique** identifié et les **modalités** (présentiel, télétravail, convention). Si une technologie vous est inconnue, ne vous disqualifiez pas : l'offre décrit souvent un profil idéal que personne ne remplit à 100 %.

CONSEIL

Commencez tôt

Les sujets les plus intéressants et les équipes les plus structurées recrutent à l'avance. Lancer vos recherches plusieurs mois avant le début du stage vous laisse le temps d'enchaîner candidatures, entretiens et formalités administratives.

1.4 Utiliser LinkedIn comme un professionnel

Pour un recruteur, votre profil LinkedIn est souvent la première image que vous donnez. Il doit être complet, cohérent avec votre CV et orienté vers l'objectif « PFE ».

Les sept éléments à soigner

Élément	Bonne pratique
Photo	Cadrage buste, fond neutre, lumière naturelle, tenue sobre.
Titre (headline)	Évitez « Étudiant ». Préférez un titre qui indique votre cible et vos compétences.
À propos	3 à 5 phrases : qui vous êtes, vos domaines, ce que vous cherchez, vos technologies clés.
Projets et expériences	Décrivez chaque projet avec contexte, rôle, technologies, résultat. Ajoutez les liens.
Compétences	Sélectionnez les compétences réellement maîtrisées, dans l'ordre de priorité.
Formation et certifications	Établissement, filière, dates ; certifications obtenues (jamais en cours sans le préciser).
Liens	URL de profil personnalisée, dépôt GitHub, portfolio si vous en avez un.

Titre peu efficace	Titre plus efficace (exemples à adapter)
Étudiant en informatique	Étudiant ingénieur en génie logiciel | Développement web full stack (React, Node.js) | À la recherche d'un PFE
Chercheur de stage	Futur ingénieur logiciel | Java, Spring Boot, MySQL | PFE [année] en développement back-end

ATTENTION

Cohérence avant tout

Une date, un intitulé ou une compétence différents entre votre CV et votre profil créent un doute. Relisez les deux documents côte à côte avant d'envoyer une candidature.

Être actif sans être envahissant

- Suivez les entreprises et les ingénieurs de votre domaine ; commentez avec du contenu utile plutôt qu'un simple « bravo ».
- Partagez parfois un apprentissage, un projet terminé, un retour d'expérience sur un hackathon.
- Restez professionnel : pas de polémique, pas d'informations confidentielles, pas de captures de code d'entreprise.

1.5 Contacter les recruteurs

Un message ciblé et respectueux vaut mieux qu'un envoi en masse. Identifiez la bonne personne : le recruteur RH, mais aussi un responsable technique, un ancien stagiaire de votre établissement ou un ingénieur de l'équipe visée.

Anatomie d'un bon message

1. **Une accroche personnalisée**
   Montrez que vous avez regardé : un projet de l'entreprise, une publication, un point commun (établissement, événement).
2. **Qui vous êtes, en une phrase**
   Formation, niveau, domaine, période de stage visée.
3. **Ce que vous demandez, précisément**
   Un échange de quelques minutes, un conseil, ou la transmission de votre candidature au bon interlocuteur.
4. **Une sortie facile**
   Remerciez, rendez la réponse simple : « même une réponse courte m'aiderait ».

Exemples de messages

Les messages de candidature complets (LinkedIn et e-mail) figurent dans la Partie 2. Voici d'autres situations courantes.

MESSAGE À UN ANCIEN STAGIAIRE DE VOTRE ÉTABLISSEMENT

Bonjour [Prénom],

Je suis étudiant en [filière] à [établissement] et je recherche un PFE dans le domaine [domaine]. J'ai vu que vous avez effectué votre stage chez [entreprise] : cette expérience m'intéresse beaucoup.

Auriez-vous quelques minutes pour me dire comment s'est déroulé votre stage et ce qui, selon vous, a compté dans votre candidature ? Je comprendrais tout à fait si vous n'avez pas le temps.

Merci d'avance, et bonne continuation. — [Prénom Nom]

MESSAGE À UN RESPONSABLE TECHNIQUE

Bonjour [Monsieur / Madame Nom],

Je suis étudiant en dernière année de [filière]. J'ai découvert votre travail sur [projet ou technologie] et je souhaite proposer ma candidature pour un PFE autour de [thème].

Mon parcours comprend [un projet pertinent en une phrase]. Pourriez-vous m'indiquer si votre équipe accueille des stagiaires de fin d'études, ou à qui je peux adresser mon dossier ?

Je vous remercie pour votre temps. — [Prénom Nom]

REMERCIEMENT APRÈS UN ÉCHANGE OU UN ENTRETIEN

Bonjour [Prénom / Madame / Monsieur],

Je vous remercie pour le temps que vous m'avez consacré [aujourd'hui / le date]. Notre échange sur [sujet précis] a renforcé mon intérêt pour le poste et pour votre équipe.

Je reste à votre disposition pour tout complément d'information. Bien cordialement, — [Prénom Nom]

1.6 Envoyer une candidature par e-mail

L'e-mail est souvent le premier document lu : son objet, sa clarté et ses pièces jointes comptent autant que votre CV.

À faire	À éviter
• Adresse e-mail sobre et professionnelle (prenom.nom). • Objet explicite : « Candidature PFE — Développement [domaine] — [Prénom Nom] ». • Un message court (5 à 8 lignes) qui renvoie aux pièces jointes. • CV en PDF nommé `CV_Nom_Prenom.pdf`. • Une signature avec téléphone, LinkedIn et GitHub. • Relire et envoyer un test à soi-même.	• Objet vide ou « Candidature » seule. • Message copié-collé pour toutes les entreprises. • Fautes d'orthographe, nom du recruteur erroné. • Pièces jointes volumineuses ou en format non lisible partout. • Nom d'entreprise d'une autre candidature oublié dans le texte. • Ton familier ou trop long récit de vie.

Le modèle complet d'e-mail de candidature, avec lettre de motivation et CV en pièces jointes, se trouve au chapitre 2.9.

À RETENIR

Avant d'appuyer sur « Envoyer »

Le bon destinataire ? Le bon nom d'entreprise ? Les deux pièces jointes ? Le lien GitHub fonctionne ? Si une seule réponse est « non », corrigez avant d'envoyer.

1.7 Relancer et organiser son suivi

Une absence de réponse ne signifie pas toujours un refus : les messages se perdent, les recruteurs sont occupés. Une relance polie, au bon moment, est une pratique professionnelle normale.

Un calendrier de relance raisonnable

Étape	Moment suggéré	Contenu
Envoi	Jour J	Candidature personnalisée.
Première relance	Environ 7 à 10 jours après	Rappel bref de la candidature + une information utile (nouveau projet, certification).
Seconde relance	Environ 2 semaines après la première	Message très court, proposition d'échange téléphonique.
Clôture	Sans réponse	Passez à autre chose, gardez le contact pour plus tard.

Ces délais sont des repères issus de la pratique, pas des règles : adaptez-les aux délais indiqués dans l'offre.

MODÈLE DE RELANCE

Objet : Relance — Candidature PFE [domaine] — [Prénom Nom]

Bonjour [Madame / Monsieur Nom],

Je me permets de revenir vers vous au sujet de ma candidature pour un PFE, envoyée le [date]. Je reste très motivé par [sujet ou équipe] et je serais ravi d'échanger avec vous quelques minutes si cela est possible.

Depuis mon premier message, j'ai [avancé sur / terminé] [élément pertinent]. Je vous joins à nouveau mon CV.

Je vous remercie pour votre attention. Bien cordialement, — [Prénom Nom]

Organiser son suivi

Tenez un tableau unique de toutes vos candidatures : entreprise, contact, canal, date d'envoi, relance, statut et prochaines actions. Il évite d'oublier une relance ou de contacter deux fois la même personne. Un modèle prêt à imprimer est fourni dans l'Outil 3.

Statut	Signification	Action suivante
À contacter	Entreprise identifiée, pas encore de message.	Préparer une candidature personnalisée.
Envoyé	Candidature transmise.	Noter la date de relance.
En discussion	Réponse reçue, échange ou entretien prévu.	Préparer l'entretien (Partie 3).
Proposition reçue	Sujet ou offre à valider.	Comparer, vérifier les conditions et la convention.
Refus	Réponse négative.	Remercier, demander un retour si possible, conserver le contact.

À RETENIR

Ce que vous retenez

Cherchez par plusieurs canaux, avec une liste cible et une routine hebdomadaire.

Personnalisez chaque message : qualité avant quantité.

Relancez une ou deux fois, poliment, et tenez un suivi écrit de chaque candidature.

M.D.S

PARTIE 2

Construire un CV et une lettre de motivation

2

Structurer un CV informatique lisible, valoriser ses projets sans rien inventer, éviter les erreurs éliminatoires et rédiger une lettre personnalisée.

AU PROGRAMME

2.1 Les sections d'un CV informatique

2.2 Présenter ses compétences techniques

2.3 Valoriser ses projets académiques

2.4 Présenter ses certifications

2.5 Choisir un design professionnel

2.6 Les erreurs qui éliminent une candidature

2.7 Exemple de CV pour un étudiant en informatique

2.8 Rédiger une lettre de motivation personnalisée

2.9 E-mail de candidature et message LinkedIn

OUTILS ASSOCIÉS DE LA BOÎTE À OUTILS

Outil 1 — Checklist de préparation du CV · Outil 2 — Checklist de candidature

2.1 Les sections d'un CV informatique

Un CV n'est pas votre biographie : c'est un document de décision. Il doit permettre à un recruteur de comprendre en quelques secondes qui vous êtes et ce que vous savez faire.

Pour un étudiant, une page suffit généralement. La hiérarchie de l'information compte plus que la quantité : ce qui prouve vos compétences doit apparaître en premier.

Section	Contenu	Conseil
En-tête	Nom, titre visé, e-mail, téléphone, ville, liens LinkedIn et GitHub.	Le titre dit ce que vous cherchez : « Étudiant ingénieur — PFE en développement web ».
Profil	2 à 3 lignes : formation, domaine, objectif.	Pas de formules vagues (« motivé, dynamique »). Citez technologies et objectifs.
Compétences techniques	Regroupées par catégories : langages, frameworks, bases de données, outils, méthodes.	Placez-la près du haut si votre expérience est courte.
Projets	2 à 4 projets les plus pertinents, avec contexte, technologies, rôle, résultat.	Section reine d'un CV d'étudiant : soignez-la.
Expérience professionnelle	Stages d'initiation, d'été ou jobs pertinents.	Sans expérience : ne remplissez pas par du vide.
Formation	Cycle actuel et précédent, établissements, dates.	Ordre antéchronologique : le plus récent d'abord.
Certifications	Titres obtenus : intitulé, organisme, année.	Uniquement ce qui est obtenu et vérifiable.
Langues	Niveau honnête (A1 à C2 ou description simple).	Précisez si vous avez passé un test.
Engagement	Clubs, associations, hackathons, bénévolat.	Valorisez la gestion d'équipe, l'initiative.



À VÉRIFIER AUPRÈS DE VOTRE ÉTABLISSEMENT

Photo, situation personnelle, mentions

L'usage de la photo, de l'état civil ou d'autres mentions personnelles varie selon les pays, les secteurs et les entreprises. Une photo sobre est souvent acceptée en Tunisie, mais n'est pas obligatoire. N'indiquez jamais de numéro d'identité ou d'informations sensibles inutiles.

2.2 Présenter ses compétences techniques

La section « Compétences » est la plus lue et la plus mal faite. Pour être crédible, elle doit être structurée, honnête et vérifiable.

À faire	À éviter
• Regrouper par catégories (langages, front-end, back-end, bases de données, outils, méthodes). • Ordonner par pertinence pour le poste visé. • Indiquer un niveau par un mot (« courant », « notions ») plutôt que des barres de progression. • Relier chaque compétence à un projet du CV. • Ne mettre que ce que vous pouvez expliquer en entretien.	• Barres de pourcentage (80 % en Java : que signifie ce chiffre ?). • Une liste de 30 technologies jamais utilisées. • Mélanger « Word » et « Kubernetes » dans la même ligne. • Ajouter une technologie « au cas où » : elle sera testée. • Compétences obsolètes sans lien avec l'offre.

Ex

EXEMPLE

Compétences regroupées

Langages : Java (courant), JavaScript / TypeScript (courant), Python (notions), SQL

Front-end : React, HTML5, CSS3

Back-end : Node.js, Express, Spring Boot (notions)

Données : MySQL, MongoDB

Outils : Git et GitHub, Docker (notions), Postman

Méthodes : UML, Scrum

CONSEIL

Adaptez le CV à chaque offre

Lisez l'offre et placez en premier les technologies qu'elle cite, si vous les maîtrisez réellement. Cette adaptation prend 10 minutes et change la perception du recruteur.

2.3 Valoriser ses projets académiques

Les projets prouvent vos compétences mieux que n'importe quelle liste. Un bon projet répond à quatre questions : **quoi** (contexte), **comment** (technologies et rôle), **quel résultat** et **où le voir** (lien).

La formule contexte – action – technologies – résultat

Version faible	Version forte (à adapter avec vos faits)
Projet de gestion de stock en Java.	Application de gestion de stock pour [type de structure] : conception UML, API Spring Boot, base MySQL, interface React ; projet de [durée] en équipe de [n].
Participation à un projet web.	Responsable du module d'authentification (JWT, rôles) et de [fonctionnalité] dans une équipe de [n] personnes.
Utilisation de Git.	Gestion du dépôt par branches de fonctionnalités et pull requests ; revue de code entre pairs.

> **ATTENTION**
> **Ne gonflez pas votre rôle**
> En entretien, on vous demandera ce que *vous* avez fait dans un projet d'équipe. Écrivez « j'ai développé » uniquement pour ce que vous avez réalisé. « Contribué à » ou « participé à » est honnête quand le travail était collectif.

Soigner son GitHub

- Un fichier `README` clair pour chaque projet important : objectif, technologies, installation, captures d'écran.
- Des noms de dépôts explicites et un historique de commits lisible.
- Aucun secret (mots de passe, clés API) dans le dépôt : un recruteur peut le consulter.
- Épinglez les trois dépôts les plus représentatifs.

2.4 Présenter ses certifications

Une certification prouve qu'un organisme reconnu a validé un niveau de connaissance. Elle n'est utile que si elle est pertinente et réellement obtenue.

Catégorie	Exemples (à titre indicatif)	Intérêt
Cloud	Certifications d'initiation proposées par les grands fournisseurs cloud (par exemple AWS Cloud Practitioner, Microsoft Azure Fundamentals).	Montre une culture cloud de base.
Agilité	Certifications Scrum d'organismes reconnus.	Utile si votre PFE s'inscrit dans une équipe agile.
Langages et éditeurs	Certifications éditeurs (Java, bases de données, etc.).	Prouve un niveau sur une technologie précise.
Cours certifiants	Parcours en ligne avec attestation.	Montre l'autonomie ; précisez que c'est une attestation de cours.

Les certifications, leurs noms et leurs conditions évoluent : vérifiez le catalogue officiel de l'organisme avant de vous y engager.

- Écrivez l'intitulé exact, l'organisme et l'année : « [Intitulé] — [organisme], [année] ».
- Ne mentionnez pas une certification « en cours » sans le dire clairement.
- Préférez peu de certifications bien choisies à une longue liste de petites attestations.

2.5 Choisir un design professionnel

Le meilleur design est celui qu'on ne remarque pas : il rend le contenu lisible. Pour un CV d'informaticien, privilégiez la sobriété.

Critère	Recommandation
Longueur	Une page en règle générale ; deux seulement si le contenu est dense et pertinent.
Typographie	Une ou deux polices lisibles (sans empattement), corps entre 10 et 11 pt, titres plus grands.
Couleurs	Une couleur d'accent maximum, bon contraste, texte noir ou gris foncé.

Critère	Recommandation
Mise en page	Marges régulières, alignements stricts, espaces blancs. Une colonne est plus sûre pour les logiciels de tri.
Format	PDF, nommé `CV_Nom_Prenom.pdf`, texte sélectionnable (pas une image).
Outils	Word, LibreOffice, Google Docs, Canva ou LaTeX : l'outil importe peu, la lisibilité oui.

Aa

DÉFINITION

ATS (Applicant Tracking System)

Logiciel de gestion des candidatures utilisé par certaines entreprises pour classer les CV. Il lit mieux un document simple : titres standards (« Compétences », « Formation »), texte sélectionnable, pas de tableaux complexes, pas d'icônes à la place des mots.

CONSEIL

Testez votre CV

Copiez le texte de votre PDF dans un éditeur de texte : si l'ordre est cohérent, un logiciel de tri le lira aussi. Faites-le relire par une personne qui ne vous connaît pas : si elle ne retient pas vos points forts en 15 secondes, simplifiez.

2.6 Les erreurs qui éliminent une candidature

Erreur	Pourquoi c'est pénalisant	Correction
Fautes d'orthographe	Signal de manque de rigueur, surtout pour un futur ingénieur.	Relire, utiliser un correcteur, faire relire par un tiers.
CV trop long ou trop chargé	Le recruteur lit vite : l'essentiel se noie.	Garder une page, supprimer le non pertinent.
Informations inexactes	Découvertes en entretien, elles détruisent la confiance.	N'écrire que des faits vérifiables.
Compétences exagérées	Les questions techniques exposent l'écart.	Utiliser « notions » quand c'est le cas.
Adresse e-mail peu sérieuse	Mauvaise première impression.	prenom.nom@...
CV identique pour tous	Sans lien avec l'offre.	Adapter profil et ordre des compétences.
Projets sans détail	« Projet web » ne dit rien.	Contexte, technologies, rôle, résultat, lien.
Liens cassés ou GitHub vide	Preuve manquante.	Tester chaque lien avant envoi.
Nom de fichier générique	« CV final (2).pdf » est difficile à classer.	CV_Nom_Prenom.pdf
Informations confidentielles	Détails d'un ancien stage soumis à confidentialité.	Décrire le contexte sans données sensibles.

2.7 Exemple de CV pour un étudiant en informatique

Le modèle ci-dessous est une **structure à compléter** : tout ce qui est entre crochets doit être remplacé par vos informations réelles. Il ne contient aucune expérience fictive.

[Prénom NOM]

Étudiant ingénieur en génie logiciel · Recherche un PFE de [durée] à partir de [mois année]

prenom.nom@email.com
+216 XX XXX XXX · [Ville]
linkedin.com/in/[profil]
github.com/[utilisateur]

COMPÉTENCES

Langages
[Java, JavaScript / TypeScript, Python, SQL]

Front-end
[React ou Angular, HTML, CSS]

Back-end
[Node.js / Express, Spring Boot]

Bases de données
[MySQL, MongoDB]

Outils
[Git / GitHub, Docker, Postman]

Méthodes
[UML, Scrum]

CERTIFICATIONS

[Intitulé — organisme — année]
uniquement celles obtenues

LANGUES

Arabe : [niveau]
Français : [niveau]
Anglais : [niveau]

ENGAGEMENT

[Club, association, hackathon : rôle et année]

PROFIL

Étudiant en [niveau] de [filière] à [établissement], intéressé par [domaine]. J'ai conçu et développé [nombre] projets avec [technologies] et je recherche un PFE autour de [thème] pour [objectif professionnel].

PROJETS

[Nom du projet 1] — [cadre : académique / personnel], [année]
- Contexte : application de [domaine] pour [utilisateurs].
- Réalisation : [fonctionnalités] avec [technologies] ; rôle : [le vôtre].
- Qualité : [tests, authentification, déploiement — seulement si réalisés].
- Dépôt : github.com/[utilisateur]/[projet]

[Nom du projet 2] — [cadre], [année]
- Contexte et objectif en une ligne.
- Technologies et votre contribution précise.
- Résultat vérifiable : [démo, lien, retour obtenu].

FORMATION

[Cycle ingénieur / licence] en [filière] — [établissement], [années]
Baccalauréat [section] — [lycée], [année]

EXPÉRIENCE PROFESSIONNELLE

[Intitulé du stage] — [entreprise], [durée]
• Missions réellement effectuées : [verbe d'action + technologie + résultat].
Pas d'expérience ? Supprimez cette section et renforcez « Projets » et « Engagement ».



À RETENIR
Rappel

Aucune ligne de ce modèle ne doit rester telle quelle : un crochet oublié dans un CV envoyé est une erreur classique.

Pour le réutiliser, copiez le texte dans Word, Google Docs ou Canva, puis adaptez la mise en forme.

2.8 Rédiger une lettre de motivation personnalisée

La lettre répond à une question simple : **pourquoi vous, pourquoi nous, pourquoi ce sujet ?** Elle ne répète pas le CV ; elle relie votre parcours au besoin de l'entreprise. Visez une page, quatre paragraphes.

1. **Accroche**
   Pourquoi cette entreprise, ce sujet, maintenant ? Une phrase précise vaut mieux qu'un compliment général.
2. **Votre parcours utile**
   Deux éléments de votre CV (projets, compétences) en lien direct avec le sujet.
3. **Ce que vous apportez et voulez apprendre**
   Motivation concrète, attentes sur l'encadrement et les technologies.
4. **Conclusion**
   Disponibilité (dates, durée), proposition d'entretien, formule de politesse.

LETTRE DE MOTIVATION — MODÈLE À ADAPTER

[Prénom NOM]
[Adresse ou ville] · [Téléphone] · [E-mail]

[Nom de l'entreprise]
À l'attention de [Madame / Monsieur Nom, ou « Service Ressources humaines »]
[Ville], le [date]

Objet : Candidature pour un stage de fin d'études (PFE) — [domaine / intitulé du sujet]

Madame, Monsieur,

Étudiant en [niveau] de [filière] à [établissement], je recherche un stage de fin d'études d'une durée de [durée] à partir de [date]. Votre travail sur [projet, produit ou technologie de l'entreprise] a retenu mon attention, car [raison précise et sincère].

Au cours de ma formation, j'ai réalisé [projet 1], une application de [description] développée avec [technologies], où j'étais responsable de [votre rôle réel]. Ce projet m'a appris à [compétence : tests, API, conception, travail en équipe], des savoir-faire qui rejoignent les besoins de votre équipe autour de [sujet ou technologie].

Je souhaite effectuer mon PFE chez [entreprise] pour [objectif : progresser sur une architecture, découvrir un métier, travailler sur un cas réel]. Je suis rigoureux, curieux et habitué à apprendre de manière autonome, comme lorsque j'ai découvert [technologie] pour [projet].

Je serais heureux de vous présenter mon parcours et de discuter du sujet lors d'un entretien. Dans l'attente de votre réponse, je vous prie d'agréer, Madame, Monsieur, l'expression de mes salutations distinguées.

[Prénom NOM]

Fournissez une version propre à chaque entreprise : une lettre générique se repère immédiatement. Ne gardez jamais le nom d'une autre société dans le texte.

ATTENTION

À éviter dans une lettre

Recopier le CV en phrases.

Des superlatifs creux (« je suis passionné par tout ce qui touche à l'informatique »).

Critiquer un autre employeur, un enseignant ou un ancien stage.

2.9 E-mail de candidature et message LinkedIn

E-MAIL DE CANDIDATURE — MODÈLE À ADAPTER

À : [adresse du recruteur] **Pièces jointes :** CV_[Nom]_[Prenom].pdf, Lettre_[Nom]_[Prenom].pdf

Objet : Candidature PFE — Développement [domaine] — [Prénom NOM]

Bonjour [Madame / Monsieur Nom],

Étudiant en [niveau] de [filière] à [établissement], je recherche un stage de fin d'études de [durée] à partir de [date], dans le domaine [domaine]. Votre offre « [intitulé de l'offre] » correspond à mon projet, notamment grâce à [technologie ou aspect précis].

J'ai réalisé [projet pertinent en une phrase], avec [technologies]. Vous trouverez mon CV et ma lettre de motivation en pièces jointes ; mon GitHub est disponible ici : [lien].

Je serais ravi d'échanger avec vous et reste à votre disposition pour un entretien, en visioconférence ou sur site.

Bien cordialement,
[Prénom NOM] · [Téléphone] · [Lien LinkedIn]

MESSAGE LINKEDIN — DEMANDE DE CONTACT

Bonjour [Prénom],

Étudiant en [filière] à [établissement], je recherche un PFE en [domaine]. Votre parcours chez [entreprise] m'a intéressé, notamment [élément précis]. Seriez-vous ouvert à un bref échange, ou pourriez-vous m'orienter vers la bonne personne ? Merci d'avance !

Un message de connexion doit rester court : une idée, une demande, un remerciement.

À RETENIR

Ce que vous retenez

Un CV d'une page, factuel, adapté à l'offre ; des projets détaillés ; aucune information inexacte.

Une lettre personnalisée qui relie votre parcours au sujet.

Des messages courts, sans fautes, avec les bons liens.

M.D.S

PARTIE 3

Réussir son entretien et intégrer l'entreprise

3

Préparer un entretien technique et RH, se présenter avec clarté, parler de ses projets, gérer l'inconnu, puis s'intégrer et communiquer avec son encadrant.

AU PROGRAMME

3.1 Préparer son entretien

3.2 Se présenter en une à deux minutes

3.3 Les questions RH classiques

3.4 Les questions techniques possibles

3.5 Parler de ses projets universitaires

3.6 Quand on ne connaît pas une technologie

3.7 Simulation d'entretien commentée

3.8 Les comportements professionnels à adopter

3.9 Communiquer avec son encadrant et comprendre les attentes

OUTILS ASSOCIÉS DE LA BOÎTE À OUTILS

Outil 4 — Checklist de préparation à l'entretien

3.1 Préparer son entretien

Un entretien se prépare comme un projet : on étudie le besoin, on prépare ses preuves, on répète, puis on s'adapte.

Les quatre axes de préparation

Axe	Ce que vous préparez
L'entreprise	Activité, produits, clients, actualités, valeurs, stack technique visible (site, offres, publications). Notez trois éléments qui vous intéressent vraiment.
Le sujet de PFE	Problème à résoudre, technologies, livrables attendus. Préparez des questions précises.
Vous-même	Votre pitch de présentation, vos projets, vos forces et vos axes de progrès, expliqués avec des exemples.
La technique	Révision des bases (POO, SQL, API REST, Git, structures de données) et des technologies citées dans l'offre.

La méthode STAR pour structurer vos exemples

Aa

DÉFINITION

Méthode STAR

Façon de raconter une expérience en quatre temps : **S**ituation (le contexte), **T**âche (votre responsabilité), **A**ction (ce que vous avez fait), **R**ésultat (ce qui s'est passé, ce que vous en avez appris).

Ex

EXEMPLE

Un exemple de réponse structurée

Dans [projet], notre équipe de [n] devait livrer [fonctionnalité] en [durée] (**situation**). J'étais chargé de [partie] (**tâche**). J'ai découpé le travail en [étapes], mis en place [outil ou pratique], et demandé une revue de code à [qui] (**action**). Nous avons livré à temps et j'ai appris à [apprentissage] (**résultat**).

Entretien en visioconférence

- Testez caméra, micro, connexion et logiciel la veille ; ayez un plan B (téléphone, autre réseau).
- Cadrez à hauteur des yeux, fond neutre, lumière devant vous, notifications coupées.
- Ayez sous la main votre CV, l'offre, un stylo et un verre d'eau ; ne lisez pas vos notes.
- Regardez la caméra pour répondre, pas seulement l'écran.

3.2 Se présenter en une à deux minutes

« Parlez-moi de vous » est presque toujours la première question. Un bon pitch est un récit court qui prépare la suite de l'entretien.

Aa **DÉFINITION**

Structure « Présent – Passé – Futur »

Présent : qui vous êtes aujourd'hui (formation, domaine). Passé : ce qui vous a construit (projets, stages, apprentissages). Futur : ce que vous cherchez et pourquoi cette entreprise.

PITCH DE PRÉSENTATION — MODÈLE (environ 1 min 30)

« Bonjour, je m'appelle [Prénom NOM]. Je suis étudiant en [niveau] de [filière] à [établissement], et je m'intéresse particulièrement à [domaine].

Au cours de ma formation, j'ai réalisé plusieurs projets, dont [projet 1] avec [technologies], où j'ai pris en charge [rôle], et [projet 2], qui m'a permis de découvrir [apprentissage]. J'ai aussi [stage, hackathon, projet personnel, si c'est le cas].

Je cherche aujourd'hui un PFE sur [thème], car j'aimerais [objectif concret]. Votre entreprise m'intéresse pour [raison précise], et le sujet proposé correspond à [lien avec votre parcours]. »

Écrivez votre version, apprenez-la comme un plan (pas mot à mot) et chronométrez-vous : une minute trente est suffisante.

À faire	À éviter
• Aller du général vers le précis. • Mentionner des technologies réelles. • Terminer par ce que vous cherchez ici. • Sourire, parler calmement.	• Réciter son CV ligne par ligne. • Parler plus de 3 minutes. • S'excuser (« je n'ai pas beaucoup d'expérience »). • Évoquer des difficultés personnelles sans rapport.

3.3 Les questions RH classiques

Les questions RH évaluent votre motivation, votre maturité et votre capacité à travailler en équipe. Elles n'ont pas de « bonne » réponse unique, mais elles demandent de la cohérence et de l'honnêteté.

Question	Ce que le recruteur cherche	Piste de réponse
Pourquoi cette entreprise ?	Un intérêt réel, pas un envoi massif.	Citez un produit, un projet, une technologie ou une valeur précis de l'entreprise.
Pourquoi ce sujet de PFE ?	Compréhension du besoin et lien avec votre parcours.	Reformulez le sujet en vos mots et reliez-le à un projet réalisé.
Quels sont vos points forts ?	Conscience de soi et preuves.	Deux qualités, chacune illustrée par un exemple concret.
Quels sont vos points faibles ?	Honnêteté et capacité d'amélioration.	Un vrai point (gestion du temps, prise de parole) et ce que vous faites pour progresser.

Question	Ce que le recruteur cherche	Piste de réponse
Où vous voyez-vous dans 3 à 5 ans ?	Ambition réaliste.	Un cap professionnel (expertise technique, responsabilités) sans promesse irréaliste.
Parlez d'un échec	Capacité à apprendre.	Une situation réelle, ce que vous avez compris, ce que vous faites différemment.
Comment gérez-vous un conflit en équipe ?	Communication et esprit de groupe.	Écoute, faits plutôt que personnes, recherche d'un compromis, implication du responsable si nécessaire.
Êtes-vous disponible aux dates demandées ?	Fiabilité.	Répondez précisément (dates, contraintes d'examens) et soyez cohérent avec la convention.
Avez-vous des questions ?	Curiosité et préparation.	Posez 2 ou 3 questions utiles (voir encadré ci-dessous).

CONSEIL

Questions intelligentes à poser

Comment s'organise le travail de l'équipe au quotidien ?

Quels seraient les livrables attendus à la fin du PFE ?

Qui serait mon encadrant et à quelle fréquence ferions-nous un point ?

Quels seraient les critères de réussite du projet pour vous ?

ATTENTION

Questions à ne pas poser en premier entretien

Évitez de commencer par la rémunération ou les congés. Ils peuvent se discuter plus tard, quand l'entreprise a confirmé son intérêt, et dans le respect du cadre fixé par votre convention.

3.4 Les questions techniques possibles

Les questions techniques portent sur ce qui figure dans votre CV et sur les fondamentaux. L'interlocuteur ne cherche pas la récitation parfaite : il observe votre raisonnement, votre précision et votre honnêteté.

Domaine	Exemples de questions	Ce qui est évalué
Programmation et POO	Différence entre classe abstraite et interface ? Qu'est-ce que le polymorphisme ? Que signifie SOLID ?	Maîtrise des concepts, capacité à donner un exemple.
Algorithmique	Complexité d'une recherche dans un tableau trié ? Différence entre pile et file ?	Raisonnement, notions de base.
Bases de données	INNER JOIN et LEFT JOIN ? Qu'est-ce qu'un index ? SQL ou NoSQL, quand choisir quoi ?	Capacité à modéliser et à justifier.
Web et API	Comment fonctionne une API REST ? PUT et PATCH ? Que signifie le code 401 ou 404 ?	Compréhension du protocole HTTP.
Sécurité	Qu'est-ce qu'une injection SQL ? Comment stocker un mot de passe ?	Réflexes de sécurité de base.

Domaine	Exemples de questions	Ce qui est évalué
Git et qualité	Merge ou rebase ? À quoi servent les tests unitaires ? Comment gérez-vous un conflit Git ?	Pratiques de travail en équipe.
Architecture	Qu'est-ce qu'une architecture 3 tiers ? MVC ? Monolithe ou microservices ?	Vision d'ensemble et vocabulaire.
Vos projets	Pourquoi ce framework ? Quel a été le plus grand problème technique ? Que referiez-vous différemment ?	Compréhension réelle de ce que vous avez construit.

Quelques réponses modèles (pistes à reformuler)

EXAMINATEUR	**Quelle est la différence entre une classe abstraite et une interface (en Java) ?**
VOUS	Une interface définit un contrat : des méthodes que la classe doit fournir, et une classe peut en implémenter plusieurs. Une classe abstraite peut en plus contenir de l'état (attributs), des constructeurs et du code partagé, mais une classe n'en étend qu'une seule. Je choisis l'interface pour décrire une capacité, la classe abstraite pour factoriser un comportement commun.

EXAMINATEUR	**Quelle différence entre INNER JOIN et LEFT JOIN ?**
VOUS	INNER JOIN ne renvoie que les lignes qui ont une correspondance dans les deux tables. LEFT JOIN renvoie toutes les lignes de la table de gauche, avec des valeurs NULL quand il n'y a pas de correspondance à droite. Par exemple, pour lister tous les utilisateurs même sans ticket, j'utilise un LEFT JOIN.

EXAMINATEUR	**PUT ou PATCH pour modifier une ressource ?**
VOUS	PUT remplace la ressource entière : le client envoie sa représentation complète. PATCH applique une modification partielle, par exemple changer uniquement le statut d'un ticket. PUT est idempotent par définition ; PATCH peut l'être selon l'implémentation.

EXAMINATEUR	**Merge ou rebase ?**
VOUS	Merge conserve l'historique tel qu'il s'est produit et crée un commit de fusion. Rebase réécrit les commits de ma branche sur une nouvelle base, ce qui donne un historique linéaire. Je n'utilise jamais rebase sur une branche déjà partagée, car cela réécrit l'historique des autres.



CONSEIL

Si l'on vous donne un exercice de code

Reformulez l'énoncé et demandez les cas limites avant de coder.

Pensez à voix haute : l'interlocuteur évalue votre démarche.

Proposez d'abord une solution simple et correcte, puis améliorez-la.

Testez votre solution avec un exemple.

3.5 Parler de ses projets universitaires

C'est le moment où vous transformez un travail académique en preuve professionnelle. Pour chaque projet important, préparez une fiche de six éléments.

Élément	Question à laquelle répondre
Problème	Quel besoin ou quelle consigne ? Pour quels utilisateurs ?
Votre rôle	Quelle part avez-vous réalisée, seul ou en équipe ?
Choix techniques	Pourquoi ces technologies et pas d'autres ?
Difficulté principale	Quel obstacle, comment l'avez-vous traité ?
Résultat	Qu'est-ce qui fonctionnait à la fin ? Comment l'avez-vous vérifié ?
Retour critique	Que feriez-vous différemment aujourd'hui ?

Ex

EXEMPLE

Formulations utiles

« J'ai choisi [technologie] parce que [raison], même si [limite]. »

« La principale difficulté était [problème]. J'ai résolu cela en [démarche], et j'ai appris que [leçon]. »

« Dans l'équipe, j'étais responsable de [partie]. Pour [autre partie], nous avons décidé ensemble de [décision]. »

« Avec plus de temps, j'aurais ajouté [amélioration] car [raison]. »

ATTENTION

Soyez prêt à descendre dans le détail

Un projet que vous ne pouvez pas expliquer jusqu'au code est un risque : si une partie a été réalisée par un coéquipier, dites-le et montrez que vous la comprenez.

3.6 Quand on ne connaît pas une technologie

Personne ne connaît tout, et un recruteur le sait. Ce qui est évalué, c'est la manière dont vous réagissez à l'inconnu. Reconnaître une limite avec calme est un signe de maturité professionnelle.

1. **Reconnaître sans dramatiser**
   « Je n'ai pas utilisé [technologie] directement. »
2. **Relier à ce que vous connaissez**
   « Je connais [technologie proche], donc je pense que le principe est [hypothèse]. »
3. **Raisonner à voix haute**
   Faites une proposition prudente, en précisant ce dont vous n'êtes pas sûr.
4. **Montrer comment vous apprendriez**
   « Je commencerais par la documentation officielle et un petit prototype. »

5 **Donner un exemple d'apprentissage rapide**
Citez une technologie découverte seul et utilisée dans un projet.

Phrases utiles	Phrases à éviter
• « Je ne l'ai pas pratiqué, mais voici comment je l'aborderais. » • « Je crois que… mais je préfère vérifier dans la documentation. » • « Pouvez-vous me donner un exemple de cas d'usage ? »	• « Je connais » alors que ce n'est pas vrai. • « Ce n'est pas utile. » • « Je ne sais pas » suivi d'un silence. • Changer de sujet sans répondre.

3.7 Simulation d'entretien commentée

Voici un entretien type, avec des réponses qui servent de **modèles de structure**. Remplacez les crochets par votre histoire : n'apprenez pas ces réponses par cœur.

RECRUTEUR	**Pouvez-vous vous présenter ?**
VOUS	Bonjour, je suis [Prénom], étudiant en [filière] à [établissement], passionné par [domaine]. J'ai réalisé [projet] avec [technologies], et je cherche un PFE sur [thème] pour [objectif]. Votre sujet m'intéresse parce que [raison].
RECRUTEUR	**Pourquoi avoir choisi notre entreprise ?**
VOUS	J'ai suivi votre travail sur [projet ou produit]. Votre équipe utilise [technologie], que j'ai déjà pratiquée dans [projet] ; j'aimerais l'approfondir sur un cas réel. J'ai aussi apprécié [valeur ou pratique].
RECRUTEUR	**Parlez-moi d'un projet dont vous êtes fier.**
VOUS	**Situation :** [projet], pour [utilisateurs], en [durée]. **Rôle :** j'étais responsable de [partie]. **Actions :** j'ai [conçu / développé / testé] avec [technologies]. **Résultat :** [ce qui fonctionnait]. **Apprentissage :** [leçon].
RECRUTEUR	**Pourquoi avoir choisi cette technologie plutôt qu'une autre ?**
VOUS	Pour [raison : écosystème, compétences de l'équipe, besoin de performance, durée du projet]. L'alternative [autre technologie] aurait été pertinente pour [cas], mais elle impliquait [contrainte].
RECRUTEUR	**Comment expliquez-vous une API REST à quelqu'un de non technique ?**
VOUS	C'est comme un serveur de restaurant : l'application cliente passe commande avec une requête standard, la cuisine (le serveur) prépare, et la réponse revient dans un format que tout le monde comprend, souvent du JSON.
RECRUTEUR	**Que faites-vous lorsque vous êtes bloqué sur un problème ?**
VOUS	Je reproduis le problème, je lis le message d'erreur complet, je cherche dans la documentation, puis je réduis le cas. Si je suis bloqué depuis [durée raisonnable, par exemple 30 à 45 minutes], je prépare une question claire (contexte, ce que j'ai essayé) et je la pose à mon encadrant.

RECRUTEUR	**Avez-vous déjà utilisé Docker ?**
VOUS	Pas en production. Je l'ai utilisé pour [usage] / Je connais le principe : un conteneur embarque l'application et ses dépendances. Je suis prêt à l'approfondir : c'est utile pour le déploiement.

RECRUTEUR	**Comment gérez-vous un désaccord technique avec un collègue ?**
VOUS	J'explique mon point de vue avec des faits (performance, maintenance, coût), j'écoute ses arguments, et nous testons si possible les deux options sur un petit cas. Si nous ne tombons pas d'accord, nous demandons l'avis de l'encadrant.

RECRUTEUR	**Avez-vous des questions ?**
VOUS	Oui : comment l'équipe organise-t-elle ses sprints ? Quels seraient les premiers livrables du PFE ? Et comment se déroulerait l'accompagnement dans les premières semaines ?

3.8 Les comportements professionnels à adopter

Une fois le sujet validé, votre comportement compte autant que votre technique. Une entreprise garde en mémoire les stagiaires fiables.

Domaine	Bonnes pratiques
Ponctualité et présence	Arriver à l'heure, prévenir en cas de retard ou d'absence, respecter les horaires et le télétravail convenus.
Communication	Messages clairs, réponses rapides, signalement précoce des blocages, ton respectueux.
Initiative mesurée	Proposer des solutions, mais demander l'accord avant de modifier le périmètre.
Confidentialité	Ne jamais partager code, données ou documents sans autorisation ; respecter les accords de confidentialité.
Sécurité	Respecter les règles d'accès, ne pas contourner les protections, ne pas installer d'outils non validés.
Tenue et image	Adaptée à la culture de l'entreprise ; professionnalisme en visioconférence aussi.
Apprentissage	Prendre des notes, poser des questions pertinentes, accepter les retours sans se défendre.

Les premiers jours

CHECKLIST — INTÉGRATION	
ADMINISTRATIF	
☐	Convention de stage signée (ou document équivalent exigé par votre établissement)
☐	Horaires, lieu, modalités de télétravail et règles internes compris
ACCÈS ET OUTILS	
☐	Poste de travail, comptes et droits d'accès reçus
☐	Environnement de développement installé et fonctionnel
☐	Accès au dépôt de code, à la documentation, au suivi des tâches
RELATIONS	
☐	Encadrant professionnel et équipe identifiés
☐	Rythme des points avec l'encadrant fixé
☐	Interlocuteurs métier identifiés

3.9 Communiquer avec son encadrant et comprendre les attentes

Votre encadrant est votre premier allié, mais il est souvent occupé. Une communication régulière, concise et orientée solutions lui permet de vous aider efficacement.

Clarifier les attentes dès le début

1. **Objectifs :** quel problème résoudre, avec quels critères de réussite ?
2. **Livrables :** application, documentation, présentation, rapport ?
3. **Périmètre :** ce qui est inclus et ce qui ne l'est pas.
4. **Contraintes et rituels :** technologies imposées, sécurité, fréquence des points, dates clés.

Un point d'avancement efficace

Rubrique	Contenu
Fait	Ce qui a été terminé depuis la dernière fois, avec une preuve (démo, capture, commit).
En cours	Ce sur quoi vous travaillez et l'avancement estimé.
Blocages	Ce qui vous freine, ce que vous avez déjà essayé, l'aide demandée.
Prévu et décisions	Objectifs de la période suivante et choix qui demandent l'avis de l'encadrant.

À RETENIR

Ce que vous retenez

Préparez l'entretien comme un projet ; racontez vos projets avec la méthode STAR, sans exagération.

Face à l'inconnu : reconnaître, raisonner, apprendre. Face à l'encadrant : précision et régularité.

M.D.S

PARTIE 4

Organiser et planifier son projet PFE

4

Transformer un besoin flou en projet cadré : problématique, objectifs, cahier des charges, user stories, méthode agile, planning et suivi.

AU PROGRAMME

4.1 Comprendre le besoin et formuler la problématique

4.2 Définir les objectifs et le périmètre

4.3 Préparer un cahier des charges

4.4 Rédiger des User Stories

4.5 Utiliser Scrum et Kanban

4.6 Répartir les tâches

4.7 Établir un planning sur 4 à 6 mois

4.8 Gérer les réunions et les comptes rendus

4.9 Travailler avec un encadrant professionnel

OUTILS ASSOCIÉS DE LA BOÎTE À OUTILS

Outil 5 — Modèle de cahier des charges · Outil 6 — Modèle de planning PFE · Outil 7 — Tableau Kanban

4.1 Comprendre le besoin et formuler la problématique

Un PFE réussi commence par une bonne compréhension du problème, avant toute ligne de code.

Un sujet de PFE est souvent exprimé comme une solution (« développer une application de… »). Votre premier travail est de retrouver le **besoin** derrière cette solution : quel problème l'entreprise veut-elle résoudre, pour qui, et comment saura-t-elle que c'est résolu ?

Les questions à poser dès les premiers jours

Thème	Questions
Contexte	Quel est le métier concerné ? Comment le travail se fait-il aujourd'hui ? Quels outils sont déjà utilisés ?
Utilisateurs	Qui utilisera la solution ? Combien sont-ils ? Quel est leur niveau technique ?
Problème	Qu'est-ce qui ne fonctionne pas ? Quelles pertes de temps, erreurs ou risques en résultent ?
Contraintes	Technologies imposées, sécurité, intégration avec l'existant, délais, budget.
Succès	Comment mesurera-t-on la réussite ? Quels sont les livrables attendus ?



CONSEIL

La technique des « 5 pourquoi »

Face à une demande, demandez « pourquoi ? » plusieurs fois de suite jusqu'à atteindre la cause réelle. « Il faut un tableau de bord » → « pourquoi ? » → « pour suivre les retards » → « pourquoi ? »… vous découvrez ce que le tableau de bord doit réellement montrer.

Formuler la problématique

La problématique est la **question centrale** à laquelle votre projet répond. Elle suit un enchaînement logique : **contexte → constat → problème → question**.

Formulation faible	Formulation solide (exemple)
Comment développer une application de tickets ?	Comment centraliser et suivre les demandes de support internes de [entreprise] afin de réduire le temps de traitement et d'éviter la perte d'informations ?
Utiliser la technologie X dans l'entreprise.	Dans quelle mesure l'adoption de [technologie] peut-elle améliorer [indicateur] dans [processus] ?



ATTENTION

Piège fréquent

Une problématique qui contient déjà la solution (« comment développer une application… ») ne montre pas que vous avez analysé le besoin. Partez du problème, pas de l'outil.

4.2 Définir les objectifs et le périmètre

Aa **DÉFINITION**

Critères SMART

Un objectif est **S**pécifique (précis), **M**esurable (on peut vérifier), **A**tteignable (réaliste), **R**éaliste au regard des ressources, **T**emporel (daté).

Objectif vague	Objectif SMART (exemple à adapter)
Améliorer le suivi des demandes.	Permettre à un employé de créer un ticket et d'en suivre le statut en temps réel, d'ici la fin du sprint 3.
Rendre l'application rapide.	Afficher la liste des tickets en moins de [durée] pour [volume] de données de test, mesuré lors de la recette.
Sécuriser le système.	Mettre en place l'authentification et les rôles (employé, technicien, admin) avant la mise en test, avec tests d'accès automatisés.

Délimiter le périmètre

Le périmètre précise ce que le projet **inclut** et **n'inclut pas**. Il protège votre planning : sans lui, les demandes s'ajoutent jusqu'à rendre le projet impossible.

Dans le périmètre	Hors périmètre (à valider)
• Authentification et gestion des rôles • Création, suivi et traitement des tickets • Tableau de bord de base • Documentation technique et guide utilisateur	• Application mobile native • Intégration avec tous les systèmes existants • Notifications par SMS • Migration complète des anciennes données

Les éléments « hors périmètre » ne sont pas supprimés : ils deviennent des **perspectives** que vous pourrez présenter dans votre rapport et votre soutenance.

4.3 Préparer un cahier des charges

Aa **DÉFINITION**

Cahier des charges

Document qui décrit ce que le système doit faire (exigences fonctionnelles), avec quelles qualités (exigences non fonctionnelles) et dans quelles limites (contraintes, livrables, planning). Il sert de référence commune entre vous, l'encadrant et l'entreprise.

Rubrique	Contenu à renseigner
Contexte et enjeux	Entreprise, activité, situation actuelle, problème rencontré.
Objectifs	Objectifs généraux et spécifiques (SMART).
Parties prenantes	Commanditaire, utilisateurs, encadrants, rôles de chacun.
Périmètre	Inclus / exclu.

Rubrique	Contenu à renseigner
Exigences fonctionnelles	Liste numérotée (EF-01, EF-02…) avec priorité.
Exigences non fonctionnelles	Performance, sécurité, ergonomie, disponibilité, maintenabilité, compatibilité.
Contraintes	Technologiques, organisationnelles, réglementaires, délais.
Livrables	Application, code source, documentation, rapport, présentation.
Critères d'acceptation	Comment le commanditaire validera chaque exigence.
Risques	Principaux risques, probabilité, impact, mesures prévues.
Planning	Phases, jalons, dates clés.

Prioriser avec la méthode MoSCoW

Catégorie	Signification	Exemple
Must have	Indispensable : sans cela, le projet n'a pas de sens.	Authentification, création et suivi de ticket.
Should have	Important, mais le projet reste utilisable sans.	Filtres et recherche.
Could have	Souhaitable si le temps le permet.	Export PDF, notifications e-mail.
Won't have (now)	Exclu pour cette version.	Application mobile native.

Ex

EXEMPLE

Exigences fonctionnelles numérotées

EF-01 (Must) — Un utilisateur authentifié peut créer un ticket avec titre, description et priorité.

EF-02 (Must) — Un technicien peut changer le statut d'un ticket qui lui est affecté.

ENF-01 (Must) — Les mots de passe sont stockés sous forme hachée.

EF-07 (Could) — L'administrateur peut exporter la liste des tickets au format CSV.

Un modèle à remplir est fourni dans l'Outil 5. **À faire valider** par votre encadrant professionnel avant de passer à la conception.

4.4 Rédiger des User Stories

Aa

DÉFINITION

User Story

Courte description d'une fonctionnalité du point de vue de l'utilisateur, sous la forme : « En tant que *[rôle]*, je veux *[action]* afin de *[bénéfice]*. » Elle se complète par des critères d'acceptation.

Ex

EXEMPLE

User Story et critères d'acceptation

En tant qu'employé, **je veux** créer un ticket de support **afin de** signaler une panne sans passer par e-mail.

Étant donné que je suis connecté, **quand** je remplis le titre et la description et que je valide, **alors** le ticket est créé avec le statut « Ouvert » et je le vois dans ma liste.

Étant donné que je suis connecté, **quand** je valide sans titre, **alors** un message d'erreur m'indique le champ manquant.

Aa

DÉFINITION

Critères INVEST

Une bonne story est **I**ndépendante, **N**égociable, apporte de la **V**aleur, est **E**stimable, **S**uffisamment petite (réalisable en quelques jours) et **T**estable.

- Formulez la valeur : si le « afin de » est vide, la story est peut-être inutile.
- Découpez les stories trop grosses (« gérer les tickets ») en étapes livrables (« créer », « consulter », « modifier le statut »).
- Écrivez les critères d'acceptation avant de coder : ils deviennent vos tests de recette.

4.5 Utiliser Scrum et Kanban

Les méthodes agiles organisent le travail en petits cycles qui produisent des résultats vérifiables, plutôt qu'un long développement suivi d'une grande surprise finale.

Scrum

Aa

DÉFINITION

Scrum

Cadre de travail agile décrit dans le « Scrum Guide ». Le travail est organisé en **sprints** (cycles de durée fixe, d'un mois maximum) qui produisent un **incrément** livrable. Trois rôles : Product Owner (porte le besoin et priorise), Scrum Master (aide l'équipe à appliquer la méthode), Développeurs.

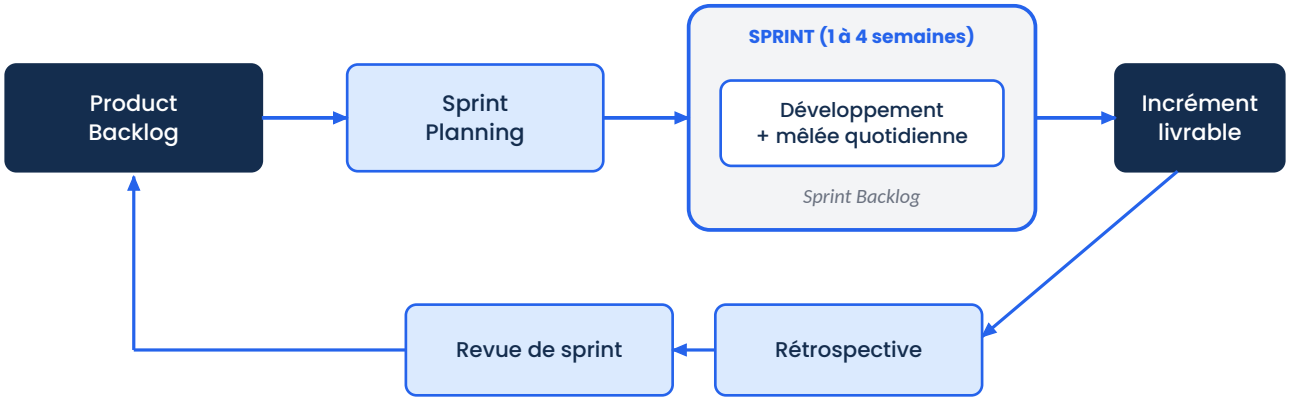


Figure 2 — Le cycle Scrum : du Product Backlog à l'incrément livrable.

Élément	Rôle	Adaptation à un PFE
Product Backlog	Liste priorisée de tout ce qui est à faire (stories).	Construisez-le avec l'encadrant professionnel ; revoyez-le chaque semaine.
Sprint Planning	Choisir ce qui sera livré dans le sprint.	Sprints courts de 2 semaines, planification en 30 à 45 minutes.
Daily Scrum	Point quotidien bref.	Individuel : un point écrit (fait / prévu / blocage) dans votre carnet.
Sprint Review	Montrer l'incrément.	Démonstration à l'encadrant en fin de sprint.
Rétrospective	Améliorer la façon de travailler.	15 minutes : que garder, arrêter, essayer ?

Kanban

Aa

DÉFINITION

Kanban

Méthode de gestion visuelle du flux de travail : les tâches progressent de colonne en colonne (À faire, En cours, En test, Terminé) et le travail en cours est **limité** (WIP, Work In Progress) pour éviter de tout démarrer sans rien terminer.

Critère	Scrum	Kanban
Rythme	Sprints de durée fixe.	Flux continu.
Planification	Engagement sur le contenu du sprint.	On tire la tâche suivante quand une place se libère.
Changement	Évité pendant le sprint.	Possible à tout moment.
Idéal quand	Le besoin est riche et doit être livré par incréments.	Le travail arrive sans cesse (corrections, support).

CONSEIL

Scrumban : le compromis pour un PFE

Beaucoup de stagiaires combinent sprints de deux semaines et tableau Kanban avec limite de travail en cours. C'est simple, visuel et facile à présenter en soutenance. Si votre entreprise impose une méthode, suivez la sienne.

4.6 Répartir les tâches

Même en PFE individuel, découper le travail est essentiel. Si vous êtes en binôme, la répartition doit être explicite pour éviter les doublons et les oublis.

1. **Découper**
   Passez des grandes fonctionnalités aux tâches de 0,5 à 2 jours (sous-tâches techniques : modèle, API, interface, tests).

2 **Estimer**
Estimez en jours ou en points relatifs ; ajoutez une marge (souvent 20 à 30 %) pour l'imprévu. Ces marges sont des repères personnels.

3 **Prioriser**
Traitez d'abord ce qui est indispensable (Must) et ce qui est risqué (nouvelle technologie, intégration) afin de lever les incertitudes tôt.

4 **Affecter**
Une tâche, un responsable. En binôme, répartissez selon compétences et besoins d'apprentissage, et faites relire le code de l'autre.

5 **Suivre**
Mettez à jour le tableau chaque jour ; tout retard important se discute avec l'encadrant.

Outil	Usage
Trello, GitHub Projects, Jira, Azure DevOps, Notion	Tableaux Kanban, backlog, sprints. Si l'entreprise a déjà un outil, utilisez-le.
Tableur	Suivi simple des tâches, estimations, risques.
Calendrier	Réunions et jalons.

4.7 Établir un planning sur 4 à 6 mois

Le planning relie les phases du projet aux dates imposées (début du stage, dépôt du rapport, soutenance). Voici un exemple pour un stage de six mois (24 semaines) : adaptez-le à votre durée.

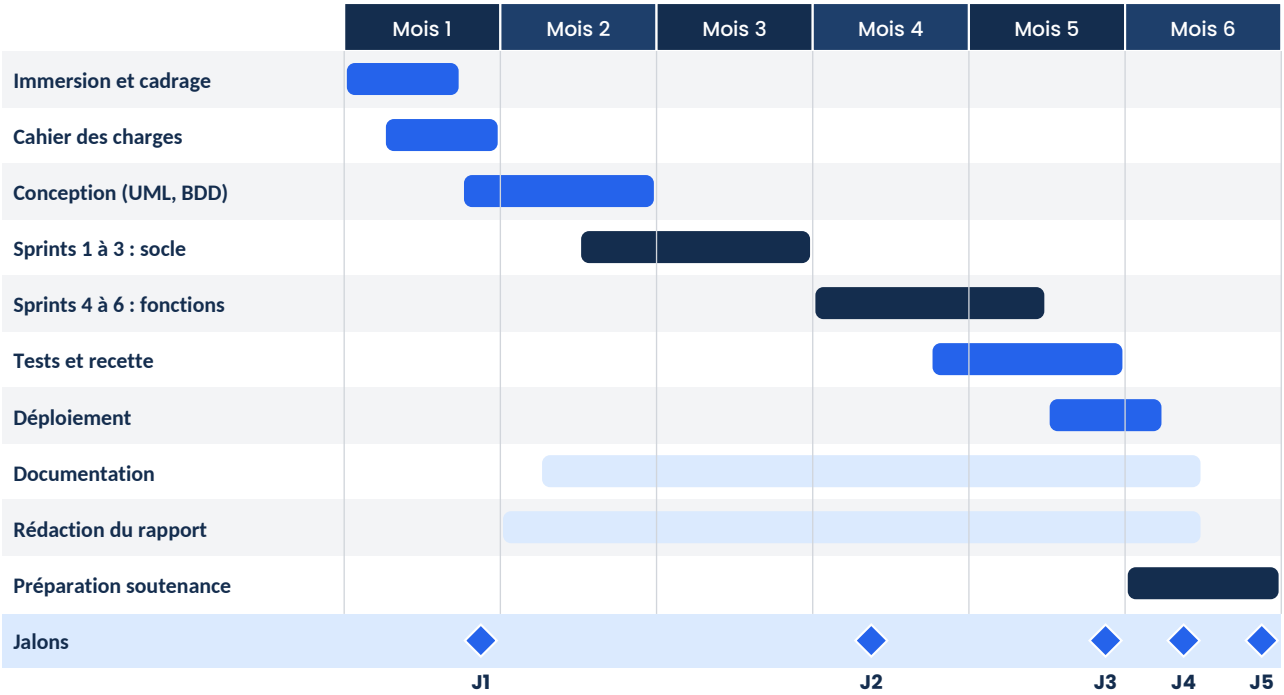


Figure 3 — Exemple de diagramme de Gantt pour un PFE de six mois (24 semaines).

Phase	Semaines	Livrables principaux
Immersion et cadrage	1 à 3	Environnement installé, besoin compris, première version de la problématique.
Cahier des charges	2 à 4	Exigences validées (jalon J1).
Conception	4 à 8	Diagrammes UML, modèle de données, architecture, maquettes.
Développement par sprints	7 à 18	Incréments testables ; démonstration intermédiaire (J2, vers S14).
Tests et recette	16 à 20	Plan de tests, corrections, version finale (J3).
Déploiement et documentation	19 à 22	Application déployée, documentation technique et utilisateur.
Rapport	5 à 22	Rédaction continue ; dépôt (J4).
Préparation de la soutenance	21 à 24	Présentation, démonstration, répétitions (J5).

Et pour quatre mois ?

Conservez les mêmes phases en les comprimant : cadrage et cahier des charges en 2 semaines, conception en 3 semaines, 4 à 5 sprints, tests intégrés à chaque sprint, rapport commencé dès la première semaine. Réduisez le **périmètre** (déplacez des fonctions en « Could » ou « Won't »), jamais les tests ni la documentation.

À VÉRIFIER AUPRÈS DE VOTRE ÉTABLISSEMENT

Dates imposées

Dates de dépôt du rapport, de pré-soutenance et de soutenance sont fixées par votre établissement. Reportez-les dans votre planning dès le premier jour et ajoutez une marge de sécurité avant chaque échéance.

4.8 Gérer les réunions et les comptes rendus

Réunion	Fréquence conseillée	Objectif
Point avec l'encadrant professionnel	Hebdomadaire (ou plus au début)	Valider l'avancement, lever les blocages, ajuster les priorités.
Démonstration (sprint review)	Fin de chaque sprint	Montrer ce qui fonctionne et recueillir des retours.
Suivi avec l'encadrant académique	Selon les règles de l'établissement	Vérifier la qualité académique, préparer le rapport.
Réunion de lancement	Une fois	Partager objectifs, périmètre et règles de collaboration.

COMPTE RENDU DE RÉUNION — MODÈLE

Réunion : [titre] **Date :** [jj/mm/aaaa] **Durée :** [x min]
Participants : [noms et rôles]

Ordre du jour : 1. [point] 2. [point] 3. [point]

Décisions : • [décision 1] • [décision 2]

Actions : [Quoi] — [Qui] — [Pour quand] / [Quoi] — [Qui] — [Pour quand]

Points ouverts / risques : [élément] **Prochaine réunion :** [date]

Envoyez le compte rendu dans les 24 heures : il évite les malentendus et constitue une trace utile pour votre rapport.

4.9 Travailler avec un encadrant professionnel

- **Autonomie progressive :** au début, demandez beaucoup de cadrage ; ensuite, arrivez avec des solutions déjà réfléchies.
- **« Pas de surprise » :** annoncez un retard dès qu'il apparaît, avec une proposition de rattrapage.
- **Changements de périmètre :** toute nouvelle demande est enregistrée, estimée, puis placée dans le backlog avec l'accord de l'encadrant ; dites ce qui sera repoussé.
- **Trace écrite :** confirmez par e-mail les décisions importantes.
- **Désaccord technique :** exposez vos arguments avec des faits, proposez un test comparatif, respectez la décision finale.
- **Double encadrement :** informez régulièrement l'encadrant académique, et ne diffusez rien de confidentiel sans accord.

À RETENIR

Ce que vous retenez

Comprendre le besoin avant de choisir la solution ; problématique, objectifs SMART et périmètre explicites.

Cahier des charges validé, user stories testables, méthode agile adaptée.

Planning visuel, réunions régulières avec comptes rendus, communication sans surprise.

M.D.S

PARTIE 5

Conception et développement technique

5

Choisir une stack adaptée, concevoir l'architecture, modéliser en UML, construire une API sécurisée et livrer un logiciel testé, versionné et documenté.

AU PROGRAMME

5.1 Choisir une stack adaptée au projet

5.2 Architecture d'une application et modèle client-serveur

5.3 Front-end et back-end

5.4 Bases de données relationnelles et non relationnelles

5.5 Modélisation UML

5.6 Concevoir une API REST

5.7 Authentification et autorisation

5.8 Sécurité informatique

5.9 Git et GitHub

5.10 Tests logiciels

5.11 Déploiement et documentation

5.1 Choisir une stack adaptée au projet

Aa **DÉFINITION**

Stack technique

Ensemble des technologies utilisées pour construire une application : langage, framework front-end, framework back-end, base de données, outils de déploiement.

Il n'existe pas de « meilleure » stack : il existe une stack **adaptée** à un besoin, à des contraintes et à une équipe. Votre rôle est de savoir justifier votre choix, car le jury le demandera presque toujours.

Critère	Question à se poser
Besoin fonctionnel	Application web, mobile, temps réel, traitement de données, API pour d'autres systèmes ?
Contraintes de l'entreprise	Technologies déjà utilisées, compétences de l'équipe, politique de sécurité, hébergement, licences.
Vos compétences et la durée	Pouvez-vous être productif rapidement ? Le temps d'apprentissage rentre-t-il dans le planning ?
Écosystème et maintenance	Documentation, communauté, bibliothèques, mises à jour, facilité de recrutement pour l'entreprise.
Performance et échelle	Charge attendue, volumes de données, besoin de temps réel.
Sécurité	Mécanismes d'authentification disponibles, maturité des bibliothèques de sécurité.

Exemple de stack	Composition	Quand y penser
MERN	MongoDB, Express.js, React, Node.js	Application web avec données flexibles ; un seul langage (JavaScript / TypeScript) sur toute la chaîne.
MEAN	MongoDB, Express.js, Angular, Node.js	Même logique avec Angular, apprécié pour sa structure imposée en équipe.
Java / Spring	Spring Boot, MySQL ou PostgreSQL, front Angular ou React	Contexte d'entreprise, applications structurées, fortes exigences de robustesse.
Classique relationnel	Back-end au choix + MySQL	Données très structurées, transactions, reporting.

ATTENTION

Évitez le « CV driven development »

Choisir une technologie uniquement parce qu'elle est à la mode, ou pour l'ajouter à son CV, met le projet en danger. Une technologie de plus doit apporter une valeur réelle au besoin.

CONSEIL

Préparez une phrase de justification

Format : « J'ai choisi [technologie] parce que [critère 1] et [critère 2], malgré [limite]. L'alternative [autre] aurait été pertinente si [condition]. »

5.2 Architecture d'une application et modèle client-serveur

DÉFINITION

Architecture logicielle

Organisation d'un système en composants et des relations entre eux. Une bonne architecture facilite les évolutions, les tests et la compréhension du code.

DÉFINITION

Architecture client-serveur

Un **client** (navigateur, application mobile) envoie des requêtes à un **serveur** qui les traite et renvoie des réponses. Le protocole du web est HTTP (HTTPS lorsqu'il est chiffré).

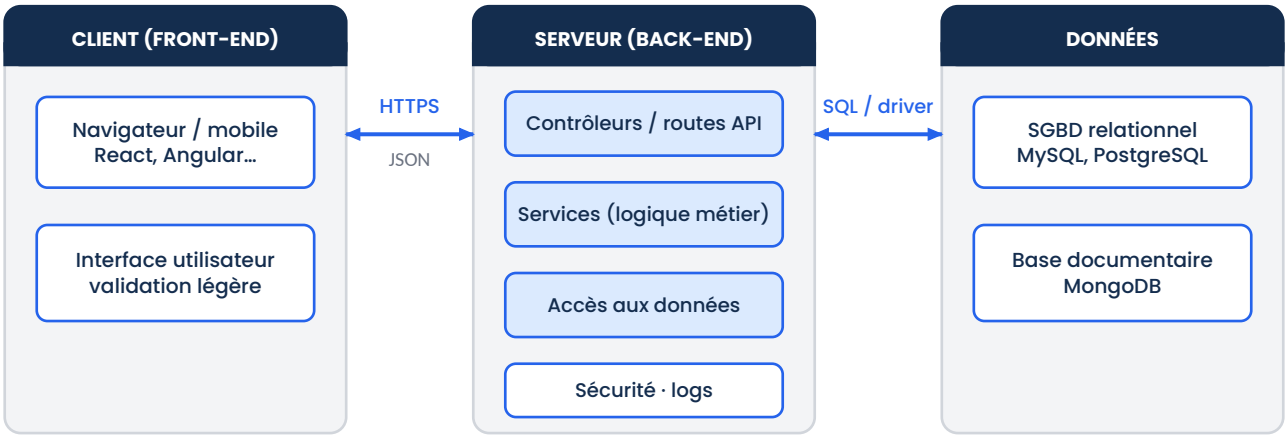


Figure 4 — Architecture en trois niveaux : présentation, logique métier, données.

Dans un back-end bien organisé, on sépare les responsabilités : le **contrôleur** reçoit la requête et valide son format, le **service** applique les règles métier, le **dépôt (repository)** dialogue avec la base. Cette séparation rend le code testable et lisible.

Style	Principe	Pour un PFE
Monolithe modulaire	Une seule application déployable, organisée en modules clairs.	Souvent le meilleur choix : simple à développer, tester et déployer.
Microservices	Plusieurs services indépendants qui communiquent par API ou messages.	À justifier : complexité accrue (réseau, déploiement, supervision). Pertinent seulement si le besoin l'exige.
MVC	Séparation Modèle, Vue, Contrôleur.	Patron très répandu pour structurer le code.

CONSEIL

Simplicité d'abord

Un monolithe propre et bien testé impressionne davantage qu'une architecture de microservices bancale. Complexifiez uniquement quand le besoin le justifie, et expliquez pourquoi.

5.3 Front-end et back-end

	Front-end	Back-end
Rôle	Interface utilisateur, interactions, affichage des données.	Règles métier, sécurité, accès aux données, API.
Technologies courantes	HTML, CSS, JavaScript / TypeScript, React, Angular.	Node.js et Express.js, Java et Spring Boot, et d'autres.
Qualités attendues	Ergonomie, accessibilité, adaptation aux écrans, validation de saisie.	Fiabilité, sécurité, performance, journalisation.

React et Angular en bref

	React	Angular
Nature	Bibliothèque pour construire des interfaces à base de composants.	Framework complet (routage, formulaires, injection de dépendances inclus).
Langage	JavaScript ou TypeScript.	TypeScript.
Philosophie	Flexible : vous choisissez les bibliothèques complémentaires.	Structuré : conventions imposées, bien adapté aux grandes équipes.
Notions à connaître	Composants, props, état (state), hooks, appels API.	Composants, modules, services, injection de dépendances, observables.

Express.js et Spring Boot : un même cas, deux approches

Voici la création d'un ticket dans les deux écosystèmes. Le principe est identique : recevoir une requête, valider, déléguer au service, répondre avec le bon code HTTP.

NODE.JS / EXPRESS.JS

```
router.post('/api/tickets', authenticate, async (req, res) => {
  const { titre, description, priorite } = req.body;
  if (!titre || !description) {
    return res.status(400).json({ message: 'Titre et description requis' });
  }
  const ticket = await ticketService.creer({
    titre, description, priorite, auteurId: req.user.id
  });
  res.status(201).json(ticket);
});
```

JAVA / SPRING BOOT

```
@RestController
@RequestMapping("/api/tickets")
public class TicketController {
    private final TicketService service;

    public TicketController(TicketService service) { this.service = service; }

    @PostMapping
    public ResponseEntity<TicketDto> creer(@Valid @RequestBody TicketRequest req) {
        TicketDto cree = service.creer(req);
        return ResponseEntity.status(HttpStatus.CREATED).body(cree);
    }
}
```

5.4 Bases de données relationnelles et non relationnelles

Aa

DÉFINITION

Base de données relationnelle

Données organisées en **tables** (lignes et colonnes) reliées par des **clés** (primaires et étrangères) et interrogées en SQL. Elle garantit l'intégrité des données et les transactions (propriétés ACID).

Aa

DÉFINITION

Base de données non relationnelle (NoSQL)

Famille de bases qui n'utilisent pas le modèle de tables : documents (MongoDB), clé-valeur, colonnes, graphes. Elles offrent souvent plus de flexibilité de schéma.

Critère	Relationnel (MySQL, PostgreSQL)	Documentaire (MongoDB)
Structure	Schéma strict, tables liées.	Documents JSON flexibles.
Relations	Jointures SQL, intégrité référentielle.	Données imbriquées ou références.
Cohérence	Transactions classiques (ACID).	Transactions possibles, modèle différent selon les cas.
Cas adaptés	Données structurées, reporting, règles strictes.	Données variables, catalogues, journaux.
Risque	Évolution du schéma plus formelle.	Cohérence à gérer dans l'application ; données dupliquées.

CONSEIL

Comment choisir ?

Si vos données ont des relations claires et des règles d'intégrité (utilisateurs, tickets, commentaires), commencez par le relationnel. Choisissez NoSQL quand la flexibilité du schéma ou le volume de données variable est un vrai besoin, pas par habitude.

MYSQL — MODÈLE RELATIONNEL

```
CREATE TABLE utilisateur (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  nom VARCHAR(100) NOT NULL,
  email VARCHAR(150) NOT NULL UNIQUE,
  mot_de_passe_hash VARCHAR(255) NOT NULL,
  role ENUM('EMPLOYE','TECHNICIEN','ADMIN') NOT NULL
);

CREATE TABLE ticket (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  titre VARCHAR(150) NOT NULL,
  description TEXT NOT NULL,
  statut ENUM('OUVERT','EN_COURS','RESOLU','CLOS') NOT NULL DEFAULT 'OUVERT',
  auteur_id BIGINT NOT NULL,
  technicien_id BIGINT NULL,
  date_creation TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (auteur_id) REFERENCES utilisateur(id),
  FOREIGN KEY (technicien_id) REFERENCES utilisateur(id)
);
```

MONGODB — DOCUMENT

```
{
  "titre": "Écran noir au démarrage",
  "statut": "OUVERT",
  "auteur": { "id": 12, "nom": "Employé A" },
  "commentaires": [
    { "contenu": "Poste redémarré, problème persistant", "date": "2026-03-04T09:12:00Z" }
  ]
}
```

Notions à maîtriser : clés primaires et étrangères, normalisation (éviter la redondance, jusqu'à la 3e forme normale), index (accélèrent les recherches mais ralentissent les écritures), jointures et transactions. Selon votre établissement, la modélisation peut se faire en **Merise** (MCD, MLD, MPD) ou en **UML** : suivez la méthode demandée.

5.5 Modélisation UML

Aa

DÉFINITION

UML (Unified Modeling Language)

Langage graphique normalisé pour décrire un système logiciel. UML 2 définit 14 types de diagrammes, mais un PFE n'en utilise en pratique que quatre à six. Le fil rouge de cette partie est une application de **helpdesk interne** : des employés créent des tickets, des techniciens les traitent.

Diagramme de cas d'utilisation

Il montre **qui** (les acteurs) utilise le système pour faire **quoi** (les cas d'utilisation). C'est le diagramme de dialogue avec le client : lisible sans connaissance technique.

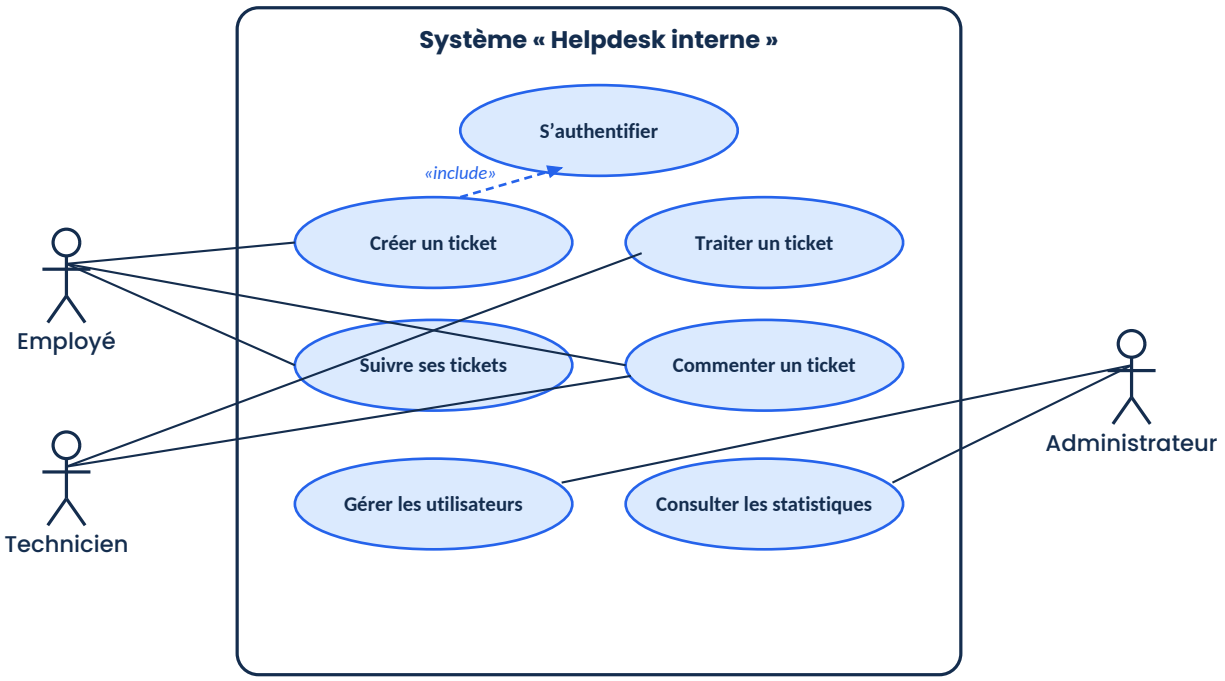


Figure 5 — Diagramme de cas d'utilisation du helpdesk.

- Un cas d'utilisation = un objectif de l'acteur, nommé par un verbe à l'infinitif.
- « include » : un cas en inclut obligatoirement un autre (créer un ticket exige de s'authentifier).
- Ne représentez pas les écrans ou les détails techniques.

Diagramme de classes

Il décrit la structure statique : les classes, leurs attributs, leurs méthodes et les relations (associations, multiplicités). Il sert de base au modèle de données et au code.

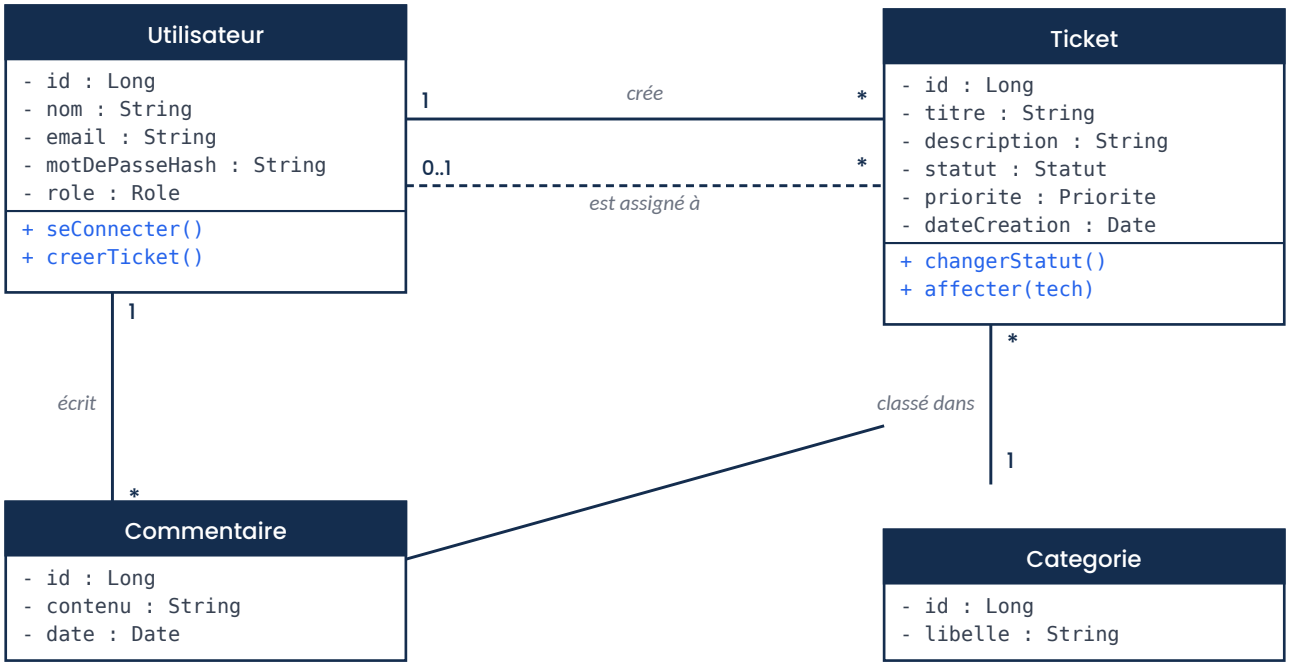


Figure 6 — Diagramme de classes du helpdesk (extrait).

- Multiplicités : 1 (exactement un), 0..1 (optionnel), * (plusieurs).
- Nommez les classes au singulier et les associations par un verbe.
- Un attribut « mot de passe en clair » n'a pas sa place : stockez un hash.

Diagramme de séquence

Il illustre les **échanges de messages dans le temps** pour un scénario précis. Il est idéal pour expliquer à un jury comment les composants collaborent.

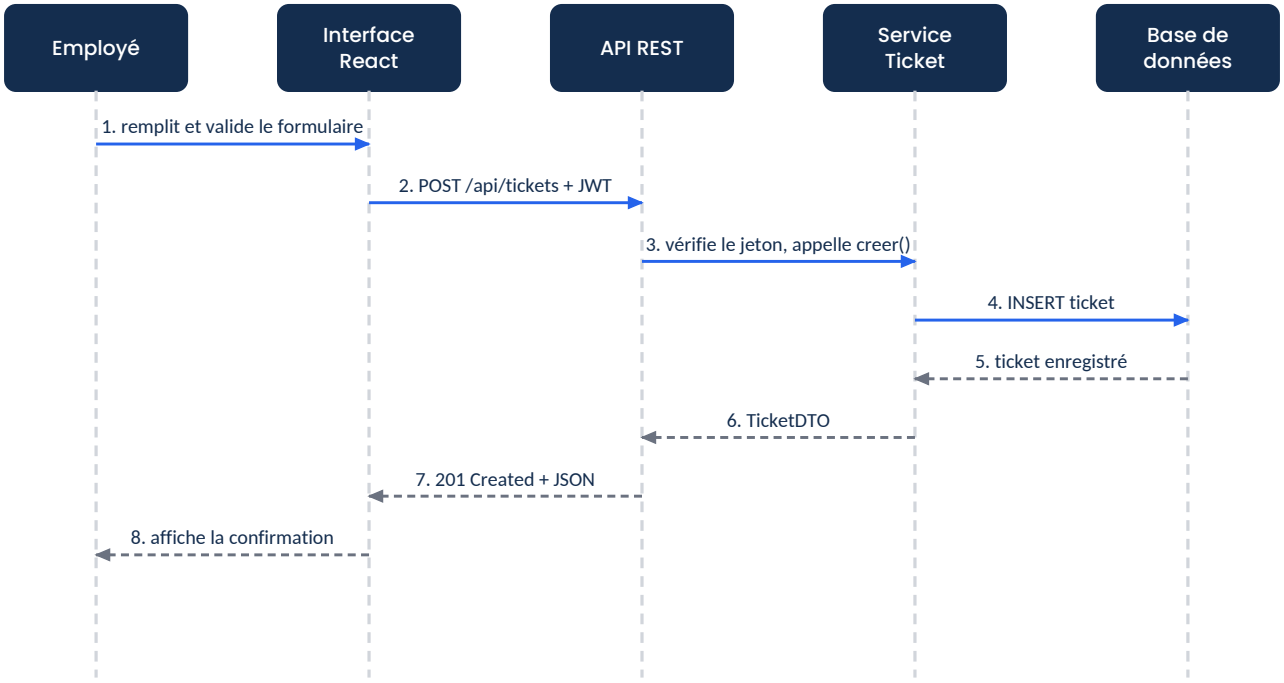


Figure 7 — Diagramme de séquence : création d'un ticket.

Diagramme d'activité

Il décrit un processus ou un algorithme : actions, décisions, boucles, parallélisme. Il convient aux règles métier (cycle de vie d'un ticket) et aux workflows.

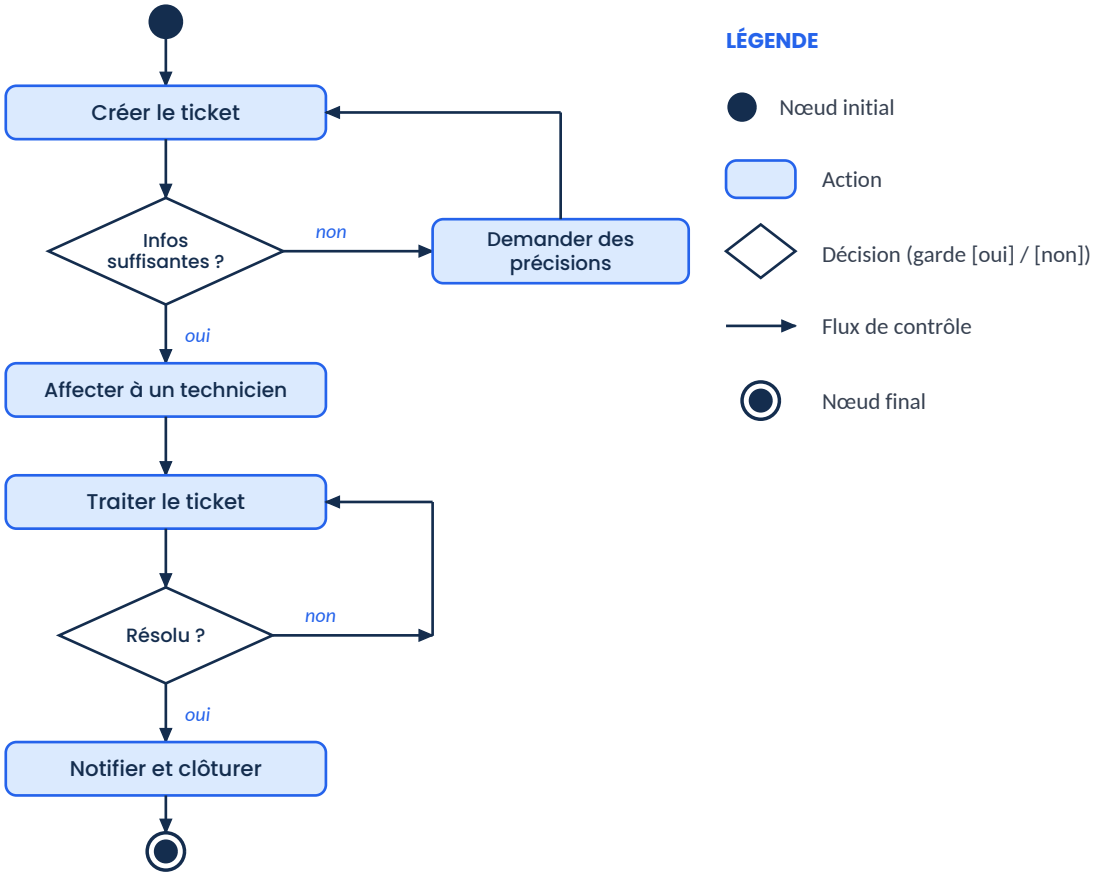


Figure 8 — Diagramme d'activité : traitement d'un ticket.

CONSEIL

Quels diagrammes présenter dans le rapport ?

Cas d'utilisation (besoins), classes (structure), séquence (scénarios clés), activité (processus), et si utile composants ou déploiement (architecture). Chaque diagramme doit être **commenté** : une figure sans explication ne démontre rien. Des outils comme draw.io, StarUML ou PlantUML permettent de les réaliser.

5.6 Concevoir une API REST

DÉFINITION

API REST

Interface de programmation qui expose des **ressources** (tickets, utilisateurs…) identifiées par des URL et manipulées avec les verbes HTTP. REST (REpresentational State Transfer) est un style d'architecture décrit par Roy Fielding dans sa thèse de doctorat en 2000. Les échanges se font généralement en JSON.

Verbe	Action	Exemple	Succès
GET	Lire	GET /api/tickets	200 OK
POST	Créer	POST /api/tickets	201 Created
PUT	Remplacer	PUT /api/tickets/42	200 OK

Verbe	Action	Exemple	Succès
PATCH	Modifier partiellement	PATCH /api/tickets/42	200 OK
DELETE	Supprimer	DELETE /api/tickets/42	204 No Content

Code	Signification	Quand l'utiliser
200 / 201 / 204	Succès / créé / pas de contenu	Réponses réussies.
400	Requête invalide	Données mal formées ou manquantes.
401	Non authentifié	Jeton absent ou invalide.
403	Interdit	Authentifié mais sans droit.
404	Introuvable	La ressource n'existe pas.
409	Conflit	Doublon ou état incompatible.
500	Erreur serveur	Erreur inattendue côté serveur.

Les bonnes pratiques

- **Noms de ressources au pluriel** et sans verbe : `/api/tickets`, pas `/api/getTickets`.
- **Pagination et filtres** pour les listes : `/api/tickets?statut=OUVERT&page=1&taille=20`.
- **Versionnement** (`/api/v1/…`) pour faire évoluer l'API sans casser les clients.
- **Format d'erreur homogène** avec un message clair, sans détails techniques sensibles.
- **Documentation** avec OpenAPI (Swagger) et collection Postman pour tester.

Endpoint	Rôle	Accès
POST /api/auth/login	Authentifier un utilisateur	Public
GET /api/tickets	Lister ses tickets (filtres)	Employé, technicien
POST /api/tickets	Créer un ticket	Employé
PATCH /api/tickets/{id}/statut	Changer le statut	Technicien affecté
POST /api/tickets/{id}/commentaires	Ajouter un commentaire	Participants
GET /api/admin/statistiques	Statistiques globales	Administrateur

5.7 Authentification et autorisation

Aa

DÉFINITION

Authentification et autorisation

Authentifier : vérifier l'identité (« qui êtes-vous ? »). **Autoriser** : décider de ce que cette identité peut faire (« avez-vous le droit ? »). Ce sont deux étapes distinctes.

Aa

DÉFINITION

JWT (JSON Web Token)

Jeton signé, décrit par la norme RFC 7519, composé de trois parties : en-tête, contenu (payload) et signature. Le serveur le vérifie sans stocker de session. Le contenu est **lisible** (simplement encodé) : n'y placez jamais de secret.

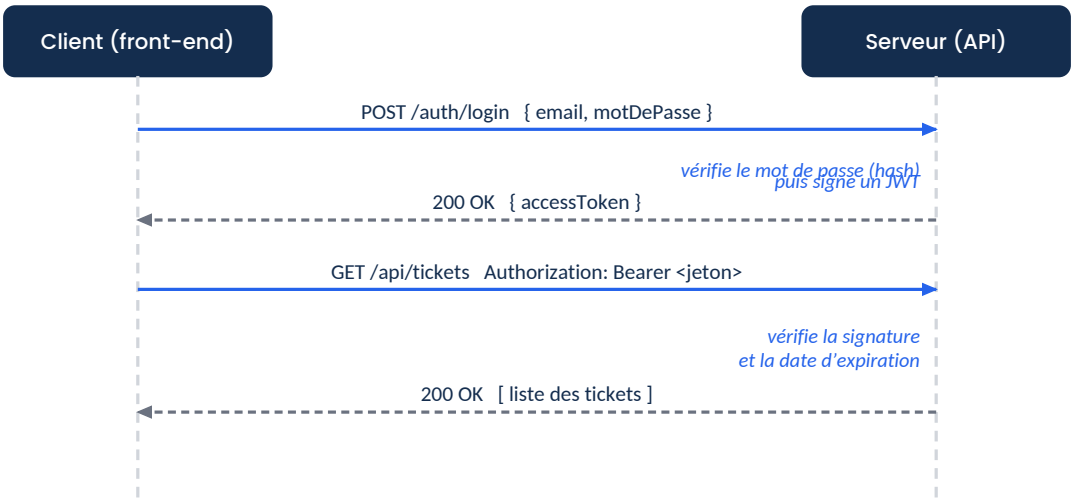


Figure 9 — Authentification par jeton JWT.

- **Mots de passe :** jamais en clair ; hachage avec un algorithme conçu pour cela (bcrypt, Argon2), avec sel.
- **Jetons :** durée de vie courte, clé de signature secrète, éventuellement jeton de rafraîchissement.
- **Autorisation par rôles (RBAC) :** chaque route vérifie le rôle ; vérifiez aussi la *propriété* (un employé ne lit que ses tickets).
- **Transport :** HTTPS obligatoire hors développement local.
- **Alternatives :** sessions côté serveur, OAuth 2.0 / OpenID Connect pour déléguer l'authentification à un fournisseur d'identité.

ATTENTION

Les contrôles côté client ne suffisent pas

Cacher un bouton dans l'interface ne protège rien. Toutes les règles d'accès doivent être vérifiées côté serveur.

5.8 Sécurité informatique

La sécurité n'est pas une étape finale : elle se pense dès la conception. Le jury et l'entreprise attendent au minimum la maîtrise des vulnérabilités courantes.

Risque (OWASP Top 10, édition 2021)	Idée de protection
A01 Contrôle d'accès défaillant	Vérifier les droits côté serveur, refus par défaut.
A02 Défaillances cryptographiques	HTTPS, hachage des mots de passe, ne pas inventer sa cryptographie.
A03 Injection	Requêtes préparées, validation et échappement des entrées.
A04 Conception non sécurisée	Modélisation des menaces dès la conception.

Risque (OWASP Top 10, édition 2021)	Idée de protection
A05 Mauvaise configuration	Désactiver les comptes et modes de debug par défaut.
A06 Composants vulnérables et obsolètes	Mettre à jour, auditer les dépendances.
A07 Identification et authentification	Mots de passe robustes, limitation des tentatives.
A08 Intégrité des logiciels et données	Vérifier les sources de dépendances et pipelines.
A09 Journalisation et supervision	Journaliser les événements de sécurité, sans données sensibles.
A10 SSRF	Valider les URL appelées par le serveur.

L'OWASP (Open Worldwide Application Security Project) publie régulièrement des mises à jour de ce classement : vérifiez l'édition en vigueur sur owasp.org.

INJECTION SQL : MAUVAISE ET BONNE PRATIQUE

```
// Dangereux : la saisie est concaténée dans la requête (injection SQL)
db.query("SELECT * FROM utilisateur WHERE email = '" + email + "'");

// Correct : requête préparée, la saisie reste une donnée
db.query("SELECT * FROM utilisateur WHERE email = ?", [email]);
```

CHECKLIST SÉCURITÉ MINIMALE

DONNÉES ET SECRETS	
☐	Aucun secret dans Git (clés, mots de passe, fichier .env ignoré)
☐	Mots de passe hachés avec bcrypt ou Argon2
☐	Données sensibles minimisées et protégées
APPLICATION	
☐	Toutes les entrées validées côté serveur
☐	Requêtes SQL préparées ; sorties échappées (XSS)
☐	Rôles et propriété des ressources vérifiés
☐	CORS restreint aux origines nécessaires
☐	Limitation des tentatives de connexion
DÉPENDANCES ET DÉPLOIEMENT	
☐	Dépendances auditées et à jour (npm audit, OWASP Dependency-Check)
☐	HTTPS activé en production
☐	Messages d'erreur sans détails internes

5.9 Git et GitHub

Aa **DÉFINITION**

Git et GitHub

Git est un logiciel de gestion de versions : il enregistre l'historique de votre code. **GitHub** est une plateforme qui héberge des dépôts Git et facilite la collaboration (pull requests, issues, intégration continue).



Figure 10 — Un flux de branches simple : fonctionnalités, intégration, version.

Commande	Rôle
git clone <url>	Copier un dépôt distant.
git status / git diff	Voir l'état et les modifications.
git switch -c feature/auth	Créer et changer de branche.
git add . / git commit -m "..."	Préparer puis enregistrer une modification.
git push / git pull	Envoyer / récupérer vers / depuis le dépôt distant.
git merge <branche>	Fusionner une branche.
git log --oneline --graph	Visualiser l'historique.
git stash	Mettre de côté des modifications en cours.

- **Messages de commit clairs**, par exemple selon la convention *Conventional Commits* : `feat(auth): ajouter la connexion JWT`, `fix(ticket): corriger le filtre par statut`.
- **Branches courtes** par fonctionnalité, fusionnées via pull request relue.
- **Un fichier .gitignore** dès le début (dépendances, fichiers de configuration locaux, secrets).
- **Étiquettes (tags)** pour marquer les versions livrées, par exemple v1.0.0 (numérotation sémantique).
- **Pousser chaque jour :** votre dépôt distant est votre sauvegarde.

ATTENTION

Ne jamais committer de secrets

Un mot de passe ou une clé ajoutés dans un commit restent dans l'historique, même supprimés ensuite. Si cela arrive, révoquez immédiatement le secret concerné.

5.10 Tests logiciels

Tester, c'est vérifier que le logiciel fait ce qu'il doit faire, et continuer à le faire après chaque modification. Les tests sont aussi un excellent argument de qualité en soutenance.

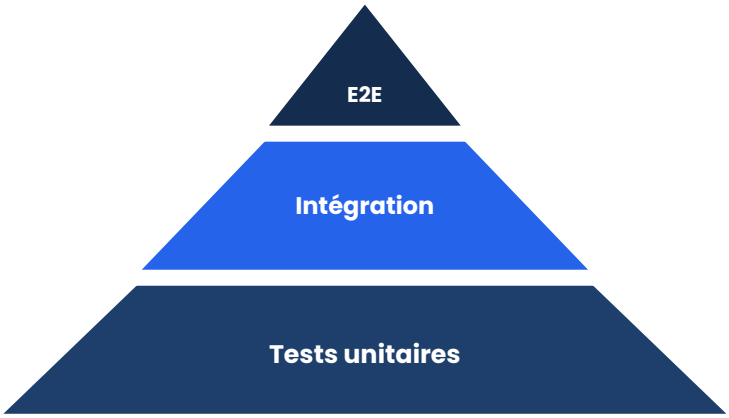


Figure 11 — La pyramide des tests : beaucoup de tests rapides à la base, peu de tests lents au sommet.

Type	Objectif	Outils courants
Tests unitaires	Vérifier une fonction ou une classe isolée.	JUnit, Mockito (Java) ; Jest (JavaScript).
Tests d'intégration	Vérifier la collaboration entre composants (API + base).	Spring Boot Test, Supertest.
Tests de bout en bout	Simuler un parcours utilisateur complet.	Cypress, Playwright.
Tests d'API	Vérifier les routes, les codes et les réponses.	Postman, Newman.
Tests non fonctionnels	Performance, sécurité, compatibilité.	Outils de charge, audit de dépendances.

JEST + SUPERTEST

```
test('refuse un ticket sans titre', async () => {
  const res = await request(app)
    .post('/api/tickets')
    .set('Authorization', `Bearer ${token}`)
    .send({ description: 'Panne' });
  expect(res.status).toBe(400);
});
```

JUNIT 5

```
@Test
void creerTicket_definitLeStatutOuvert() {
    Ticket t = service.creer(new TicketRequest("Panne", "Écran noir"));
    assertEquals(Statut.OUVERT, t.getStatut());
}
```

- Écrivez un **plan de test** relié aux exigences : identifiant, scénario, résultat attendu, résultat obtenu, statut.
- Automatisez les tests importants et lancez-les à chaque push (intégration continue).
- Mesurez la couverture à titre d'indicateur, pas comme objectif absolu : un pourcentage élevé ne garantit pas la qualité.
- Conservez les résultats et captures : ils alimentent le chapitre « Tests » de votre rapport.

5.11 Déploiement et documentation

Les environnements

Séparez le **développement** (votre machine), le **test** ou *staging* (copie proche de la production) et la **production**. La configuration (adresses, mots de passe) passe par des **variables d'environnement**, jamais par le code.

Aa

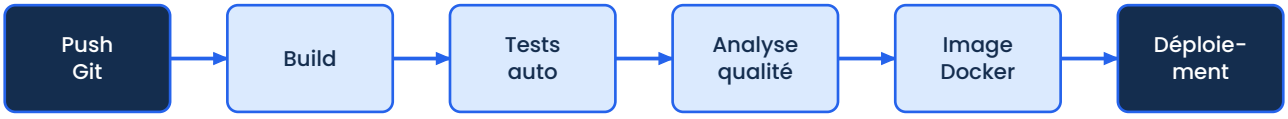
DÉFINITION

Docker

Outil qui empaquette une application et ses dépendances dans une **image** exécutable partout sous forme de **conteneur**. Il réduit les écarts entre « ça marche chez moi » et « ça marche en production ».

DOCKERFILE — EXEMPLE MINIMAL (À ADAPTER)

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
EXPOSE 3000
CMD ["node", "server.js"]
```



Chaque étape échoue « vite » : un test rouge bloque la suite du pipeline.

Figure 12 — Un pipeline d'intégration et de déploiement continus.

L'intégration continue (CI) lance automatiquement build et tests à chaque modification ; le déploiement continu (CD) livre ensuite une version validée. GitHub Actions, GitLab CI ou Jenkins permettent de le mettre en place. Si l'entreprise a déjà sa chaîne de déploiement, utilisez-la.

La documentation à produire

Document	Contenu	Public
README	Présentation, prérequis, installation, lancement, variables d'environnement, tests.	Développeurs
Documentation d'API	Routes, paramètres, réponses, exemples (OpenAPI / Swagger).	Développeurs, intégrateurs
Documentation technique	Architecture, diagrammes, modèle de données, choix et justifications.	Équipe, successeurs
Guide utilisateur	Pas à pas illustré des fonctionnalités principales.	Utilisateurs
Guide d'exploitation	Déploiement, sauvegardes, supervision, résolution des incidents courants.	Administrateurs

README.md — SQUELETTE

```
# Nom du projet
Description courte en deux phrases.

## Prérequis
Node.js [version], MySQL [version]

## Installation
1. Cloner le dépôt
2. Copier .env.example en .env et renseigner les variables
3. npm install

## Lancement et tests
npm run dev  |  npm test

## Structure du projet
src/ (routes, services, modèles), tests/, docs/
```



CONSEIL

Notez vos décisions d'architecture

Pour chaque choix important (technologie, modèle de données, méthode d'authentification), consignez en quelques lignes : le contexte, la décision, les alternatives écartées. Ces notes deviennent le chapitre « Conception » de votre rapport et préparent les questions du jury.



À RETENIR

Ce que vous retenez

Une stack se justifie par des critères, pas par la mode ; un monolithe propre vaut mieux qu'une complexité inutile.

UML, API REST, authentification et sécurité se conçoivent ensemble.

Git quotidien, tests automatisés, déploiement reproductible et documentation vivante font la différence.

M.D.S

PARTIE 6

Rédiger un rapport PFE professionnel

6

Connaître la structure complète d'un rapport, savoir ce que contient chaque chapitre, rédiger dans un style académique clair et maîtriser Word, WPS Office ou LibreOffice.

AU PROGRAMME

6.1 Avant de commencer : gabarit et méthode

6.2 La structure complète du rapport

6.3 Le contenu attendu dans chaque chapitre

6.4 Exemples de rédaction académique

6.5 Mise en page, figures, tableaux

6.6 Citer ses sources

6.7 Word, WPS Office ou LibreOffice : réussir la mise en forme

OUTILS ASSOCIÉS DE LA BOÎTE À OUTILS

Outil 8 — Checklist de rédaction du rapport

6.1 Avant de commencer : gabarit et méthode

À VÉRIFIER AUPRÈS DE VOTRE ÉTABLISSEMENT

Le gabarit officiel prime sur ce guide

Nombre de pages, plan imposé, police, marges, style de citation, langue, pages obligatoires : tout cela est fixé par votre établissement ou votre département. Demandez le gabarit et le guide de rédaction avant d'écrire la première page, et suivez-les même s'ils diffèrent des conseils ci-dessous.

- **Écrivez au fil de l'eau :** un chapitre rédigé juste après le travail correspondant est plus précis et plus rapide qu'un rapport écrit en un mois.
- **Gardez un journal de bord :** décisions, difficultés, captures d'écran, résultats de tests.
- **Faites relire par l'encadrant** chapitre après chapitre, pas seulement à la fin.
- **Sauvegardez et versionnez** le rapport (cloud, copie externe, noms de fichiers datés).

6.2 La structure complète du rapport

Voici la structure la plus courante. Les intitulés et l'ordre exacts varient : certains établissements regroupent des chapitres ou en imposent d'autres.

Élément	Fonction	Longueur indicative
Page de garde	Identifier le travail : établissement, titre, auteur, encadrants, entreprise, année.	1 page
Dédicaces	Facultatif, personnel.	½ page
Remerciements	Remercier encadrants, équipe, famille, avec sobriété.	1 page
Résumé et Abstract	Synthèse du projet en français et en anglais, avec mots-clés.	1 page chacun
Table des matières et listes	Plan, liste des figures, des tableaux, des abréviations.	2 à 4 pages
Introduction générale	Contexte, problématique, objectifs, méthode, plan du rapport.	2 à 3 pages
Chapitre 1	Présentation de l'entreprise et contexte du projet.	8 à 12 % du corps
Chapitre 2	Étude de l'existant, critique, problématique et solution proposée.	10 à 15 %
Chapitre 3	Analyse des besoins : acteurs, exigences, cas d'utilisation.	12 à 15 %
Chapitre 4	Conception : architecture, modèles UML, base de données.	15 à 20 %
Chapitre 5	Réalisation : environnement, implémentation, interfaces.	20 à 25 %
Chapitre 6	Tests et validation (parfois inclus dans la réalisation).	8 à 10 %
Conclusion et perspectives	Bilan, apport, limites, évolutions.	1 à 2 pages
Bibliographie / webographie	Sources citées.	1 à 2 pages
Annexes	Documents volumineux ou secondaires.	Variable

Les longueurs ci-dessus sont des repères personnels pour équilibrer votre rapport, pas des exigences : suivez le nombre de pages demandé par votre établissement.

6.3 Le contenu attendu dans chaque chapitre

Chapitre	Ce qu'il doit contenir	Erreur fréquente
Introduction générale	• Contexte du projet, en entonnoir (du général au particulier). • Problématique et objectifs. • Démarche et plan du rapport, chapitre par chapitre.	Raconter le stage au lieu de poser un problème.
Présentation de l'entreprise	• Activité, taille, organisation, domaine. • Service d'accueil, équipe, environnement technique. • Contexte précis du projet.	Copier la plaquette commerciale ; données confidentielles.
Étude de l'existant	• Description du système ou processus actuel. • Limites constatées, avec des exemples. • Solutions existantes comparées (benchmark). • Problématique et solution retenue.	Critiquer sans arguments ni critères.
Analyse des besoins	• Acteurs et parties prenantes. • Besoins fonctionnels et non fonctionnels. • Cas d'utilisation, user stories, priorisation. • Méthode de gestion de projet.	Lister des fonctionnalités sans les prioriser.
Conception	• Architecture générale justifiée. • Diagrammes UML commentés. • Modèle de données. • Conception de la sécurité et des interfaces (maquettes).	Diagrammes sans explications ni cohérence avec le code.
Réalisation	• Environnement matériel et logiciel, technologies justifiées. • Implémentation des parties importantes (extraits courts). • Captures d'écran commentées.	Coller des pages de code ; captures illisibles.
Tests et validation	• Stratégie et plan de tests. • Résultats, anomalies, corrections. • Validation par l'entreprise.	Affirmer que « tout fonctionne » sans preuve.
Conclusion et perspectives	• Rappel du problème et de ce qui a été réalisé. • Limites assumées. • Évolutions possibles. • Apports personnels et professionnels.	Introduire de nouveaux éléments ou se contenter d'un résumé plat.

CONSEIL

Un fil conducteur

Chaque chapitre doit se terminer par une courte conclusion qui annonce le suivant. Le lecteur doit pouvoir suivre la logique : problème, besoins, conception, réalisation, validation.

6.4 Exemples de rédaction académique

Principes de style

Style recommandé	À éviter
• Phrases courtes et précises, une idée par paragraphe. • Vocabulaire technique défini à la première occurrence. • Voix impersonnelle ou « nous » de manière cohérente. • Chaque figure introduite et commentée dans le texte. • Affirmations appuyées par des faits, des sources ou des tests.	• Phrases interminables et jargon non défini. • Superlatifs non justifiés (« très performant »). • Alternance « je » / « nous » / « on ». • Copie de documentation ou d'un autre rapport. • Fautes d'orthographe et de typographie.

Ex **EXEMPLE**

Introduction (contexte problème objectif)

Dans un contexte où les entreprises numérisent la gestion de leurs services internes, [entreprise] traite aujourd'hui les demandes de support par e-mail. Cette pratique entraîne [constat : pertes d'informations, absence de suivi]. Dans ce cadre, notre projet consiste à concevoir et développer [solution] afin de [objectif mesurable]. Ce rapport est organisé en [n] chapitres : le premier présente…

Ex **EXEMPLE**

Justification d'un choix technique

Parmi les solutions étudiées (tableau 3.2), nous avons retenu Spring Boot pour sa maturité, son intégration avec [élément] et la maîtrise de l'équipe. Angular a été écarté en raison de [contrainte] ; cette décision reste ré-évaluable si [condition].

Ex **EXEMPLE**

Remerciements (sobres)

Je tiens à exprimer ma sincère gratitude à [Nom], mon encadrant professionnel, pour sa disponibilité et ses conseils, à [Nom], mon encadrant académique, pour son suivi, ainsi qu'à toute l'équipe de [entreprise] pour son accueil.

Ex **EXEMPLE**

Résumé : cinq éléments

Contexte en une phrase, **problème**, **solution** avec technologies, **résultats**, **mots-clés**.

Abstract : *This project aims to design and develop [solution] for [company] in order to [goal]. The application, built with [technologies], …*

6.5 Mise en page, figures, tableaux

À VÉRIFIER AUPRÈS DE VOTRE ÉTABLISSEMENT

Normes typographiques

Police, taille, interligne, marges, numérotation et position des légendes sont souvent imposées. Les valeurs ci-dessous sont des usages courants à confirmer.

Aspect	Usage courant
Texte	Police lisible (par exemple Times New Roman 12 pt ou Arial / Calibri 11 pt), interligne 1,15 à 1,5, texte justifié ou aligné à gauche de façon constante.
Marges	Environ 2,5 cm de chaque côté, marge de reliure si impression.
Titres	Hiérarchie numérotée : 1, 1.1, 1.1.1 (trois niveaux maximum), police et style identiques partout.
Pagination	Chiffres romains pour les pages préliminaires (remerciements, résumés, tables), chiffres arabes à partir de l'introduction.
Figures	Légende **sous** la figure : « Figure 3.2 : Diagramme de séquence — création d'un ticket ». Numérotation par chapitre.
Tableaux	Légende souvent **au-dessus** du tableau : « Tableau 2.1 : Comparaison des solutions existantes ».
Captures d'écran	Nettes, recadrées, cohérentes, sans données personnelles ; chaque capture est commentée.
Code	Extraits courts et pertinents dans le corps ; code long en annexe ou dépôt.
Renvois	Citer chaque figure dans le texte : « comme le montre la figure 3.2… ».

CONSEIL

Cohérence

Un seul style de puces, une seule façon d'écrire les sigles, les mêmes couleurs pour les diagrammes. Le jury remarque l'uniformité.

6.6 Citer ses sources

Toute idée, définition, figure ou résultat qui ne vous appartient pas doit être attribué. Ne pas le faire est du **plagiat**, y compris pour un schéma copié ou un texte reformulé trop près de l'original.

Style	Dans le texte	Exemple de liste (référence réelle)
Numérique (IEEE)	Renvoi [1], [2]…	[1] R. T. Fielding, « Architectural Styles and the Design of Network-based Software Architectures », thèse de doctorat, University of California, Irvine, 2000.
Auteur-date (APA)	(Fielding, 2000)	Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Doctoral dissertation]. University of California, Irvine.

Style	Dans le texte	Exemple de liste (référence réelle)
Livre	Selon le style choisi	Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
Page web	Selon le style choisi	Schwaber, K., & Sutherland, J. (2020). *The Scrum Guide*. scrumguides.org (date de consultation : [jj/mm/aaaa]).

- Choisissez **un** style (ou celui imposé) et appliquez-le partout.
- Notez la **date de consultation** des ressources en ligne.
- Privilégiez documentations officielles, livres, articles scientifiques ; méfiez-vous des sources anonymes.
- Les figures empruntées portent la mention « Source : [auteur, année] ». Un logiciel comme Zotero (libre) gère vos références.

6.7 Word, WPS Office ou LibreOffice : réussir la mise en forme

Les trois logiciels permettent d'obtenir un rapport professionnel s'ils sont utilisés avec des **styles** plutôt qu'avec des mises en forme manuelles. Les noms de menus varient selon la version et la langue.

Besoin	Word	WPS Office	LibreOffice Writer
Titres hiérarchiques	Styles Titre 1, 2, 3 (onglet Accueil).	Styles de titre (interface proche de Word).	Styles de paragraphe « Titre 1, 2, 3 ».
Numérotation des titres	Liste à plusieurs niveaux liée aux styles de titre.	Numérotation multiniveau.	Outils > Numérotation des chapitres.
Table des matières	Références > Table des matières.	Références > Table des matières.	Insertion > Table des matières et index.
Légendes et listes de figures	Références > Insérer une légende, puis table des illustrations.	Références > Légende.	Insertion > Légende, puis index des illustrations.
Numérotation de pages	Sections et « lier au précédent » pour changer de format.	Sections de la même façon.	Styles de page distincts.
Renvois	Références > Renvoi.	Références > Renvoi.	Insertion > Référence croisée.
Mise à jour	Tout sélectionner puis F9.	Mettre à jour les champs.	Outils > Actualiser > Tous les index.

1. **Définir les styles avant d'écrire**
   Corps de texte, Titre 1 à 3, légende, citation, code : modifiez-les une fois, ils s'appliquent partout.
2. **Écrire sans mise en forme manuelle**
   Pas de saut de ligne pour espacer ; légendes automatiques ; table des matières et listes générées puis mises à jour avant chaque envoi.
3. **Exporter en PDF et vérifier**
   Polices, images, sauts de page, numéros, liens, accents : relisez le PDF final page par page.

M.D.S

PARTIE 7

Préparer une soutenance exceptionnelle

Construire une présentation claire, expliquer l'architecture, réussir sa démonstration, gérer temps et stress, et répondre avec assurance aux questions du jury.

AU PROGRAMME

7.1 Comprendre le format de la soutenance

7.2 Sélectionner l'essentiel

7.3 Structure type de 12 à 15 diapositives

7.4 Présenter problématique, solution et architecture

7.5 Réussir sa démonstration

7.6 Gérer le temps et le stress

7.7 Répondre aux questions du jury

7.8 30 questions possibles et pistes de réponses

OUTILS ASSOCIÉS DE LA BOÎTE À OUTILS

Outil 9 — Checklist de préparation de la soutenance · Outil 10 — Planning des dernières semaines

7.1 Comprendre le format de la soutenance

À VÉRIFIER AUPRÈS DE VOTRE ÉTABLISSEMENT

Format et critères

Durée de l'exposé, durée des questions, composition du jury (président, rapporteur, encadrants), présence de l'entreprise, langue, critères de notation : tout dépend de votre établissement. Obtenez ces informations par écrit et construisez votre présentation autour de ces contraintes.

Une soutenance évalue quatre choses : la **qualité du travail** réalisé, votre **maîtrise** technique et méthodologique, la **clarté** de votre communication et votre **recul critique**. Le jury a lu (ou survolé) votre rapport : il attend de vous une synthèse structurée, pas une récitation.

Dans la suite, nous prenons un exposé de **20 minutes** à titre d'exemple. Adaptez les durées proportionnellement.

7.2 Sélectionner l'essentiel

Le plus difficile n'est pas de remplir une présentation, mais de choisir ce qu'on laisse de côté. Posez-vous, pour chaque information : *le jury en a-t-il besoin pour comprendre le problème, la solution ou la valeur du travail ?*

À mettre en avant	À laisser au rapport
Le problème et pourquoi il compte.	L'historique détaillé de l'entreprise.
Vos décisions de conception et leur justification.	Les listes exhaustives de fonctionnalités.
Un ou deux diagrammes clés, lisibles.	Tous les diagrammes UML.
La démonstration du cœur de valeur.	Chaque écran de l'application.
Les résultats mesurés et les limites.	Les extraits de code longs.

Principes de conception des diapositives

- **Une idée par diapositive**, avec un titre qui dit le message (« L'architecture sépare… »).
- **Peu de texte :** des mots-clés et des schémas, pas des paragraphes.
- **Taille lisible** au fond de la salle : évitez les polices trop petites.
- **Cohérence visuelle :** même modèle, mêmes couleurs, même style de schéma.
- **Images nettes** et commentées oralement ; pas de capture illisible.
- Un **fil conducteur** : problème → solution → preuve.

7.3 Structure type de 12 à 15 diapositives

Voici une structure de 15 diapositives avec une durée indicative pour un exposé de 20 minutes ; la démonstration en occupe une part importante. Réduisez le contexte si le temps manque.

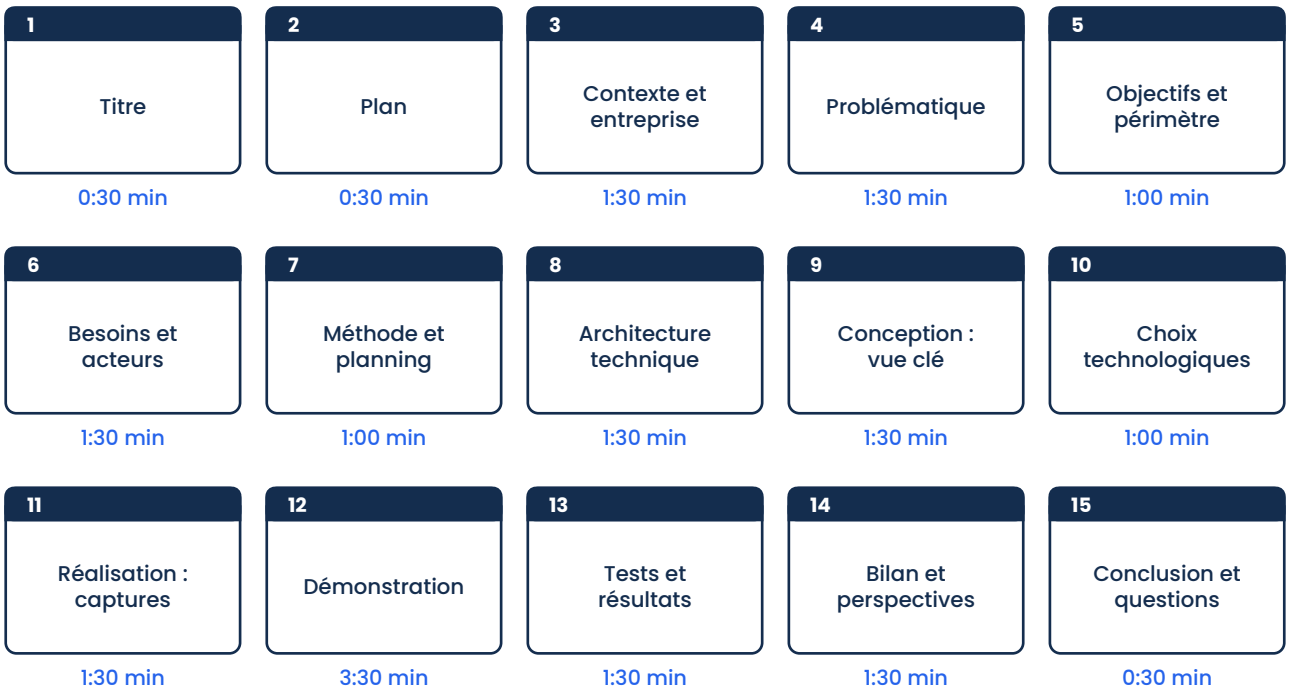


Figure 13 — Structure type de 15 diapositives avec durées indicatives.

N°	Diapositive	Contenu essentiel
1	Titre	Titre du projet, nom, établissement, entreprise, encadrants, année.
2	Plan	Quatre à six parties, sans détail.
3	Contexte et entreprise	Activité, équipe d'accueil, contexte du projet.
4	Problématique	Constat, problème, question à laquelle répond le projet.
5	Objectifs et périmètre	Objectifs SMART, inclus / exclu.
6	Besoins et acteurs	Cas d'utilisation synthétique, exigences clés.
7	Méthode et planning	Scrum ou Kanban, sprints, chronologie.
8	Architecture technique	Schéma global, flux, justification du modèle.
9	Conception : vue clé	Un diagramme (séquence ou classes) bien commenté.
10	Choix technologiques	Tableau comparatif court et décision.
11	Réalisation	Quelques captures significatives.
12	Démonstration	Scénario court, de bout en bout.
13	Tests et résultats	Stratégie, résultats, indicateurs, anomalies traitées.
14	Bilan et perspectives	Ce qui est livré, limites, évolutions possibles.
15	Conclusion et questions	Message final, remerciements.

7.4 Présenter problématique, solution et architecture

Raconter une histoire

Un bon exposé suit un enchaînement narratif : **quelle situation**, **quel problème**, **quelle idée**, **quelle solution**, **quelle preuve**. Commencez par une phrase accrocheuse qui rend le problème concret : « Aujourd'hui, [exemple vécu]. »

Expliquer l'architecture en trois niveaux

1. **La vue d'ensemble**
   Un schéma simple : utilisateurs, front-end, API, base de données, services externes. Dites en une phrase le rôle de chaque bloc.
2. **Un flux concret**
   Suivez une action (« un employé crée un ticket ») à travers les composants, en désignant les flèches.
3. **Une décision justifiée**
   Choisissez un point (sécurité, technologie, découpage) et expliquez pourquoi, avec l'alternative écartée.



CONSEIL

Zoom progressif

Ne montrez pas un schéma de 20 composants d'un coup. Affichez les blocs un à un, ou mettez en évidence la zone dont vous parlez. Si le schéma est illisible à l'écran, simplifiez-le.

7.5 Réussir sa démonstration

La démonstration est le moment où votre travail devient réel. C'est aussi le moment le plus risqué : réseau, base de données, données de test, bug de dernière minute.

CHECKLIST — DÉMONSTRATION	
SCÉNARIO	
☐	Scénario de 3 à 5 minutes écrit, avec les étapes et la valeur montrée
☐	Données de test préparées, réalistes, sans informations confidentielles
☐	Comptes de démonstration créés (plusieurs rôles si besoin)
TECHNIQUE	
☐	Application lancée et testée juste avant (local ou déployé)
☐	Plan B prêt : vidéo enregistrée ou captures d'écran
☐	Connexion internet non indispensable ou solution de secours
☐	Écran, résolution, zoom du navigateur, notifications coupées
PENDANT	
☐	Annoncer ce que l'on va montrer et pourquoi
☐	Commenter chaque action à voix haute
☐	En cas de bug : rester calme, passer au plan B

- **Ne codez pas en direct** : montrez le résultat, pas le débogage.
- Montrez le **parcours le plus important**, pas toutes les fonctions ; mentionnez les autres.
- Répétez la démonstration plusieurs fois, dans les mêmes conditions que le jour J.

7.6 Gérer le temps et le stress

Gestion du temps



Figure 14 — Répartition du temps pour un exposé de 20 minutes (regroupe les diapositives de la figure précédente).

- **Chronométrez** vos répétitions complètes, à voix haute, au moins trois fois ; enregistrez-vous.
- Prévoyez des **diapositives de repli** à sauter en cas de retard, et des diapositives « de réserve » pour les questions (en annexe).
- Surveillez l'horloge à des repères fixes (par exemple à la moitié de l'exposé).
- Ne dépassez jamais le temps : une conclusion bien tenue vaut mieux qu'une démonstration bâclée.

Maîtriser le stress

Un peu de stress est normal et peut même vous donner de l'énergie. L'objectif n'est pas de le supprimer, mais de le canaliser.

Moment	Pratiques utiles
Les jours avant	Répétitions devant un public (amis, famille, collègues), simulation de questions, sommeil régulier.
Le matin	Repas léger, arrivée en avance, test du matériel, relecture rapide du plan (pas du contenu entier).
Juste avant	Respiration lente (inspirer en comptant quatre, expirer en comptant six), un verre d'eau, posture droite.
Pendant	Première phrase préparée, voix posée, pauses entre les idées, regard vers chaque membre du jury.
Après un trou de mémoire	Silence calme, regard sur la diapositive, reprise avec le dernier mot-clé.

Si le stress devient difficile à vivre au quotidien, parlez-en à une personne de confiance ou à un professionnel : ce n'est pas un manque de compétence.

7.7 Répondre aux questions du jury

1. **Écouter jusqu'au bout**
   Notez la question ; ne coupez pas.
2. **Reformuler si nécessaire**
   « Si je comprends bien, vous demandez pourquoi… ». Cela vous laisse le temps de réfléchir.
3. **Répondre court d'abord**
   Donnez la réponse en une phrase, puis justifiez et illustrez.
4. **Rester honnête**
   « Je n'ai pas testé ce cas, mais voici comment je le traiterais. »
5. **Gérer la critique**
   Remerciez, reconnaissez ce qui est fondé, expliquez le contexte sans vous justifier excessivement.

> **ATTENTION**
> **Ce qui fait perdre des points**
> Inventer une réponse, se défendre agressivement, répondre à côté de la question, ou prétendre avoir fait ce que vous n'avez pas fait. Un jury expérimenté repère très vite le bluff.

7.8 30 questions possibles et pistes de réponses

Ces questions sont des exemples fréquents. Elles ne constituent pas une liste officielle : préparez vos **propres** réponses, adaptées à votre projet.

Contexte et problématique

N°	Question	Piste de réponse
1	Quel problème votre projet résout-il, et pour qui ?	Reformulez la problématique en une phrase avec un exemple concret et les utilisateurs concernés.
2	Pourquoi ce sujet était-il pertinent pour l'entreprise ?	Parlez des gains attendus : temps, erreurs, coûts, qualité, mais seulement ceux que vous pouvez justifier.
3	Qu'existait-il avant votre solution ?	Décrivez l'existant, ses limites, et les solutions du marché étudiées.
4	Pourquoi ne pas utiliser une solution existante ?	Critères : coût, adaptation au besoin, intégration, contrôle des données, licence.
5	Quels étaient les critères de réussite ?	Reprenez les objectifs SMART et le résultat de la recette.

Choix techniques

N°	Question	Piste de réponse
6	Pourquoi ces technologies ?	Critères (besoin, équipe, écosystème, durée), alternative écartée, limite assumée.
7	Pourquoi une base relationnelle (ou NoSQL) ?	Nature des données, relations, transactions, évolutivité du schéma.
8	Pourquoi une architecture monolithique (ou en microservices) ?	Taille du projet, complexité acceptable, équipe, déploiement.
9	Que feriez-vous si le nombre d'utilisateurs était multiplié par 100 ?	Pistes : cache, index, pagination, mise à l'échelle horizontale, supervision ; reconnaître ce qui n'a pas été testé.
10	Quelles bibliothèques tierces utilisez-vous et sont-elles fiables ?	Maintenance, licence, audit de vulnérabilités, mises à jour.
11	Quelle est la partie du code dont vous êtes le plus fier ?	Choisissez une partie utile, expliquez la difficulté et la solution.

Conception et architecture

N°	Question	Piste de réponse
12	Pouvez-vous expliquer votre diagramme de classes ?	Parlez des entités principales, de leurs relations et des règles métier qu'elles portent.
13	Comment avez-vous conçu votre API ?	Ressources, verbes, codes de statut, validation, versionnement, documentation.
14	Comment gérez-vous les erreurs ?	Validation, codes HTTP, messages clairs, journalisation, pas de détails sensibles.
15	Comment la logique métier est-elle séparée de l'interface ?	Couches (contrôleur, service, dépôt), testabilité, réutilisation.
16	Quels patrons de conception avez-vous utilisés ?	Exemples réels (MVC, dépôt, injection de dépendances) ; ne citez que ce que vous avez appliqué.

Sécurité et qualité

N°	Question	Piste de réponse
17	Comment sécurisez-vous l'application ?	Authentification, rôles, hachage, HTTPS, validation, requêtes préparées, secrets hors du dépôt.
18	Comment stockez-vous les mots de passe ?	Hachage avec sel via bcrypt ou Argon2 ; jamais en clair ni chiffrement réversible.
19	Comment avez-vous testé votre solution ?	Pyramide des tests, outils, plan de test, résultats, anomalies corrigées.
20	Quelles sont les limites de votre solution ?	Citez deux limites réelles et comment vous les traiteriez : c'est une preuve de maturité.
21	Quelles données personnelles traitez-vous ?	Minimisation, accès contrôlé, conformité aux règles de l'entreprise et à la législation applicable.

Gestion de projet et bilan

N°	Question	Piste de réponse
22	Quelle méthode de gestion de projet avez-vous utilisée ?	Scrum, Kanban ou autre, adaptation à votre contexte, outils, rituels.
23	Quel a été votre plus grand défi ?	Une difficulté réelle, la démarche de résolution, ce que vous en avez retenu.
24	Avez-vous respecté votre planning initial ?	Écarts, causes, actions correctives ; ne pas nier un retard.
25	Que referiez-vous différemment ?	Une ou deux leçons concrètes (tests plus tôt, périmètre plus étroit, meilleure documentation).
26	Quelles évolutions proposeriez-vous ?	Perspectives classées par valeur et effort, issues du « hors périmètre ».
27	Qu'avez-vous appris, techniquement et humainement ?	Deux compétences techniques, une compétence relationnelle, avec un exemple.

À RETENIR

Ce que vous retenez

Une soutenance est une histoire : problème, solution, preuve.

Moins de texte, plus de schémas, une démonstration scénarisée avec plan B.

Chronométrez, répétez, restez honnête face aux questions.

M.D.S

PARTIE 8

Les erreurs à éviter et les conseils pour se démarquer

8

Identifier les pièges classiques d'un PFE et adopter les pratiques qui distinguent un stagiaire fiable, curieux et force de proposition.

AU PROGRAMME

8.1 Les huit erreurs les plus coûteuses

8.2 Comment se démarquer pendant son stage PFE ?

8.3 Après la soutenance

8.1 Les huit erreurs les plus coûteuses

Erreur	Symptômes	Remède
Mauvaise gestion du temps	Planning jamais mis à jour, tests et documentation repoussés, nuits blanches finales.	Planning visible, jalons hebdomadaires, marge de sécurité, périmètre réduit tôt plutôt que tard.
Manque de communication	Encadrant sans nouvelles pendant des semaines, blocages cachés.	Point régulier, message d'avancement, alerte dès le premier retard.
Code non organisé	Fichiers géants, noms flous, logique dans les contrôleurs, aucun test.	Architecture en couches, conventions de nommage, revues de code, refactorisation continue.
Absence de sauvegardes	Un seul ordinateur, aucune copie du rapport ni du code.	Dépôt distant, copie externe ; règle 3-2-1 : trois copies, deux supports, une copie hors site.
Mauvaise gestion de Git	Commits énormes, « fix », branches oubliées, secrets poussés, conflits non résolus.	Commits atomiques, branches courtes, pull requests, .gitignore dès le début.
Rapport rédigé à la dernière minute	Textes copiés, schémas incohérents avec l'application, fautes.	Rédiger au fil du projet, relectures progressives, checklist (Outil 8).
Présentation surchargée	Diapositives pleines de texte, lecture à voix haute, dépassement de temps.	Une idée par diapositive, répétitions chronométrées (partie 7).
Démonstration mal préparée	Bug en direct, données vides, réseau indisponible.	Scénario écrit, jeu de données, plan B vidéo, test avant l'exposé.

ATTENTION

Le piège de la perfection

Passer des semaines sur une fonctionnalité secondaire pendant que le cœur du projet n'est pas testé est l'une des erreurs les plus courantes. Livrez d'abord une version simple qui fonctionne de bout en bout, puis améliorez-la.

8.2 Comment se démarquer pendant son stage PFE ?

Se démarquer ne signifie pas travailler plus d'heures : cela veut dire apporter de la valeur de façon fiable, visible et professionnelle.

Levier	Comment faire concrètement
Proposer des améliorations pertinentes	• Observez les frictions réelles des utilisateurs et de l'équipe. • Présentez une proposition : problème, solution, coût estimé, bénéfice. Laissez l'encadrant décider.
Faire une veille technologique	• 30 minutes par semaine : documentation officielle, newsletters et blogs d'éditeurs, conférences enregistrées, notes de version des outils que vous utilisez. • Partagez une découverte utile avec l'équipe, brièvement, en l'illustrant sur le projet.

Levier	Comment faire concrètement
Automatiser	• Scripts pour les tâches répétitives (installation, données de test, rapports de tests). • Intégration continue, formatage et lint automatiques.
Documenter	• README, procédures d'installation, décisions d'architecture : la documentation est souvent ce qui reste après votre départ.
Utiliser l'IA intelligemment	• Pour apprendre, déboguer, relire : toujours vérifier et comprendre (voir le chapitre dédié). • Respecter la politique de l'entreprise sur les outils et les données.
Faire preuve d'initiative	• Anticiper les risques, poser des questions préparées, proposer de prendre une tâche utile à l'équipe. • Initiative ne veut pas dire sortir du périmètre sans accord.
Construire une bonne réputation	• Fiabilité : tenir ses engagements et prévenir à l'avance. • Respect : écouter, remercier, accepter les retours. • Discrétion : confidentialité absolue. • Visibilité : démonstrations régulières, résultats clairs.

Les comportements qui se remarquent	Ceux qui se remarquent aussi
• Arriver avec une question précise et ce que vous avez déjà essayé. • Résumer par écrit après une réunion. • Reconnaître une erreur rapidement et proposer une correction. • Aider un collègue quand c'est possible.	• Disparaître en cas de difficulté. • Se contenter du strict minimum. • Modifier le code des autres sans prévenir. • Promettre sans pouvoir tenir.



CONSEIL

Transformer le stage en opportunité

Un stage réussi ouvre des portes : recommandations, contacts, parfois une proposition d'embauche. Rien n'est garanti et cela dépend du contexte de l'entreprise ; concentrez-vous sur la qualité de votre travail et de vos relations, et gardez le contact après votre départ.

8.3 Après la soutenance

- **Remercier** par message personnel l'encadrant professionnel, l'encadrant académique et le jury.
- **Déposer les versions finales** demandées (rapport corrigé, code, documentation), dans le respect de la confidentialité.
- **Mettre à jour** CV et LinkedIn avec le projet, les technologies et les résultats vérifiables.
- **Capitaliser :** notes de retour d'expérience, enseignements, éventuellement une démonstration publique si l'entreprise l'autorise.
- **Conserver le réseau :** un message de nouvelles quelques mois plus tard maintient la relation (et renseignez-vous sur les formalités de fin d'études propres à votre établissement).

M.D.S

CHAPITRE SPÉCIAL

L’intelligence artificielle au service du PFE

IA

Choisir et utiliser des assistants d’IA pour progresser plus vite, sans sacrifier la fiabilité, l’intégrité académique ni la confidentialité.

AU PROGRAMME

A.1 Panorama des outils

A.2 Sept usages concrets

A.3 Douze prompts prêts à copier-coller

A.4 Limites, risques et bonnes pratiques

A.1 Panorama des outils

Les assistants d'IA générative produisent du texte, du code ou des analyses à partir d'instructions en langage naturel (les **prompts**). Ils évoluent très vite : consultez toujours la documentation et les conditions d'utilisation officielles avant de choisir un outil, notamment pour les offres, les limites et la gestion des données.

Outil	Nature	Usages utiles en PFE	Points d'attention
ChatGPT (OpenAI)	Assistant conversationnel généraliste.	Explications, brainstorming, reformulation, aide au code.	Peut produire des erreurs plausibles ; vérifier.
Claude (Anthropic)	Assistant conversationnel, à l'aise avec les textes longs.	Relecture de documents, analyse de code, structuration de rapports.	Même exigence de vérification.
Gemini (Google)	Assistant conversationnel intégré à l'écosystème Google.	Synthèse, recherche d'idées, travail avec des documents Google.	Vérifier les réglages de confidentialité.
GitHub Copilot	Assistant dans l'éditeur de code.	Complétion, génération de tests, explication de code.	Code à relire ; licences et sécurité.
Cursor	Éditeur de code avec IA intégrée.	Modifications guidées sur plusieurs fichiers, refactorisation.	Risque de changements massifs non compris.
Perplexity	Moteur de réponses qui cite ses sources.	Recherche documentaire, état de l'art, comparaison de technologies.	Ouvrir les sources citées, ne pas se fier au résumé seul.

CONSEIL

Un outil ne fait pas le projet

Le plus important n'est pas l'outil, mais la qualité de votre demande et votre capacité à juger la réponse. Testez deux ou trois outils sur une même question pour comparer.

A.2 Sept usages concrets

Usage	Comment l'IA aide	Ce que vous gardez la main
Améliorer un CV	Reformuler des phrases, repérer les redondances, proposer une structure.	Les faits : aucune compétence ni expérience ne doit être ajoutée par l'IA.
Préparer un entretien	Simuler un recruteur, poser des questions, donner un retour sur vos réponses.	Votre histoire et vos mots : ne récitez pas un texte généré.
Comprendre un concept	Expliquer avec des analogies, des exemples, à plusieurs niveaux.	La vérification dans la documentation officielle.
Générer des idées de projets ou de fonctionnalités	Lister des options, des risques, des cas limites.	Le choix, selon le besoin réel de l'entreprise.
Analyser des erreurs	Expliquer un message d'erreur, proposer des pistes de diagnostic.	La reproduction du bug et le test de la correction.
Améliorer la documentation	Relire un README, uniformiser le style, générer un squelette.	L'exactitude technique.

Usage	Comment l'IA aide	Ce que vous gardez la main
Préparer une soutenance	Simuler un jury, reformuler des messages, chronométrer à partir d'un script.	Le contenu, votre expression, votre maîtrise du sujet.

A.3 Douze prompts prêts à copier-coller

Remplacez les éléments entre crochets. Un bon prompt donne un **rôle**, un **contexte**, une **tâche précise** et un **format de réponse**. N'y collez jamais de données confidentielles.

PROMPT 1 Améliorer un CV sans rien inventer

```
Agis comme un recruteur technique. Voici mon CV (texte) : [coller]. Je vise un PFE en [domaine].
Propose : 1) les points faibles, 2) des reformulations plus précises de mes projets, 3) l'ordre
idéal des sections.
Règle stricte : n'ajoute aucune compétence, expérience ou chiffre qui ne figure pas dans mon
texte ; pose-moi des questions si une information manque.
```

PROMPT 2 Simuler un entretien technique

```
Agis comme un ingénieur qui mène un entretien de PFE en [domaine, ex. développement web]. Mon
profil : [résumé]. Pose-moi une question à la fois, adaptée à mon niveau, puis attends ma
réponse. Après chaque réponse, donne : un point fort, une erreur ou omission, et une version
améliorée. Commence par la première question.
```

PROMPT 3 Comprendre un concept

```
Explique-moi [concept, ex. JWT] en trois niveaux : 1) en deux phrases pour un débutant, 2) avec
une analogie, 3) avec un exemple technique concret en [langage]. Termine par 3 erreurs fréquentes
et 2 sources officielles où vérifier.
```

PROMPT 4 Générer des idées de fonctionnalités

```
Contexte : [description du besoin de l'entreprise]. Utilisateurs : [qui]. Contraintes :
[technologies, durée].
Propose 10 fonctionnalités classées Must / Should / Could (méthode MoSCoW), avec pour chacune la
valeur apportée, la difficulté estimée (faible/moyenne/élevée) et un risque.
```

PROMPT 5 Analyser une erreur

```
Voici mon environnement : [langage, version, framework]. Voici le message d'erreur complet :
[coller]. Voici le code concerné (anonymisé) : [coller].
Explique la cause probable, propose 3 pistes de diagnostic classées par probabilité, puis la
correction. Indique comment vérifier que le problème est résolu.
```

PROMPT 6 Relire et améliorer un README

```
Voici le README de mon projet : [coller]. Évalue-le selon : clarté, installation reproductible,
exemples, structure. Propose une version améliorée en conservant strictement les informations
exactes et signale ce qui manque.
```

PROMPT 7 Préparer des questions de jury

```
Voici le résumé de mon projet de fin d'études : [coller]. Génère 15 questions qu'un jury pourrait poser (contexte, choix techniques, sécurité, tests, gestion de projet), du plus simple au plus difficile. Ne donne pas les réponses : j'y répondrai puis tu me feras un retour.
```

PROMPT 8 Structurer un plan de rapport

```
Mon PFE porte sur [sujet], réalisé chez [type d'entreprise]. Mon établissement demande [exigences connues]. Propose un plan détaillé de rapport (chapitres, sections, contenu attendu de chacun) et une liste de figures à prévoir. Signale ce que je dois vérifier auprès de mon établissement.
```

PROMPT 9 Reformuler un paragraphe académique

```
Reformule le paragraphe suivant dans un style académique clair, en français, sans changer le sens ni ajouter d'information : [coller]. Montre-moi les modifications principales et pourquoi.
```

PROMPT 10 Revoir du code

```
Voici une fonction en [langage] : [coller]. Fais une revue de code : lisibilité, bugs potentiels, cas limites, sécurité, tests manquants. Classe les remarques par importance. Ne réécris pas tout : explique d'abord pour que je comprenne.
```

PROMPT 11 Générer des cas de test

```
Voici la spécification d'une fonctionnalité : [coller user story et critères d'acceptation]. Propose un plan de test : cas nominaux, cas limites, cas d'erreur, avec données d'entrée et résultat attendu, sous forme de tableau.
```

PROMPT 12 Construire un plan de soutenance

```
J'ai [durée] pour présenter mon PFE : [résumé]. Propose une structure de [nombre] diapositives avec, pour chacune, le message principal, ce qu'il faut montrer et la durée. Indique ce que je dois laisser dans le rapport.
```

A.4 Limites, risques et bonnes pratiques

Risque	Explication	Que faire ?
Fausses informations	Les modèles peuvent produire des réponses fausses mais convaincantes (« hallucinations »), y compris des références ou des fonctions inexistantes.	Vérifier dans la documentation officielle, exécuter le code, ouvrir les sources.
Plagiat et intégrité académique	Présenter un texte généré comme le sien peut être sanctionné ; les règles varient selon l'établissement.	Connaître la politique de votre établissement, déclarer l'usage si demandé, rédiger avec vos mots.
Confidentialité	Code, données clients, architecture et documents internes peuvent être soumis à des accords de confidentialité.	Ne rien coller sans accord écrit ; anonymiser ; privilégier les outils validés par l'entreprise.

Risque	Explication	Que faire ?
Sécurité du code généré	Le code peut contenir des failles ou des dépendances risquées.	Relire, tester, auditer, appliquer la checklist sécurité.
Droits et licences	Le code généré peut ressembler à du code sous licence.	Vérifier les conditions de l'outil et de l'entreprise.
Dépendance	Accepter des réponses sans les comprendre empêche d'apprendre et fragilise la soutenance.	Demander des explications ; savoir justifier chaque ligne.

AVANT D'UTILISER UNE RÉPONSE D'IA	
CONTRÔLE	
☐	J'ai compris la réponse et je pourrais l'expliquer au jury
☐	J'ai vérifié les faits dans une source officielle
☐	J'ai exécuté et testé le code dans mon environnement
CONFORMITÉ	
☐	Aucune donnée confidentielle n'a été transmise
☐	Mon usage respecte les règles de l'établissement et de l'entreprise
☐	Les passages repris sont reformulés et, si besoin, mentionnés

À RETENIR

Principe directeur

L'IA est un excellent assistant et un mauvais responsable : c'est vous qui signez le travail, qui le comprenez et qui répondez aux questions du jury.

M.D.S

BOÎTE À OUTILS

Boîte à outils : dix ressources prêtes à l'emploi

KIT

Des checklists et modèles à imprimer ou à recopier dans votre outil habituel pour suivre votre PFE de la candidature à la soutenance.

AU PROGRAMME

1. Checklist de préparation du CV
2. Checklist de candidature
3. Tableau de suivi des entreprises contactées
4. Checklist de préparation à l'entretien
5. Modèle de cahier des charges
6. Modèle de planning PFE
7. Exemple de tableau Kanban
8. Checklist de rédaction du rapport
9. Checklist de préparation de la soutenance
10. Planning des dernières semaines avant la soutenance

1 Checklist de préparation du CV

À valider avant chaque envoi.

CV	
CONTENU	
☐	Une page (deux au maximum, si justifié)
☐	Titre visé et profil de 2 à 3 lignes
☐	Compétences regroupées par catégories, sans barres de niveau
☐	Projets : contexte, technologies, rôle, résultat, lien
☐	Aucune information inexacte ou exagérée
FORME	
☐	Police lisible, mise en page cohérente, marges équilibrées
☐	Orthographe relue par une tierce personne
☐	Export en PDF, texte sélectionnable
☐	Fichier nommé CV_Nom_Prenom.pdf
LIENS ET CONTACTS	
☐	E-mail professionnel et numéro valides
☐	LinkedIn et GitHub à jour, liens testés
☐	Aucun secret ni donnée confidentielle

2 Checklist de candidature

Avant d'appuyer sur « Envoyer ».

CANDIDATURE	
PRÉPARATION	
☐	Offre lue en entier, technologies repérées
☐	Entreprise étudiée (activité, produits, actualités)
☐	CV adapté à l'offre
ENVOI	
☐	Bon destinataire, bon nom d'entreprise, bonne civilité
☐	Objet explicite
☐	Lettre personnalisée en pièce jointe
☐	Pièces jointes ouvertes et vérifiées
SUIVI	
☐	Candidature notée dans le tableau de suivi (Outil 3)
☐	Date de relance programmée

3 Tableau de suivi des entreprises contactées

À recopier dans un tableur pour un usage durable.

Entreprise	Contact / canal	Envoi	Relance	Statut	Notes

Statuts suggérés : À contacter · Envoyé · En discussion · Entretien planifié · Proposition reçue · Refus.

4 Checklist de préparation à l'entretien

À relire la veille.

ENTRETIEN	
CONNAISSANCES	
☐	Pitch de présentation de 1 min 30 chronométré
☐	Entreprise et sujet de PFE étudiés
☐	Trois projets expliqués avec la méthode STAR
☐	Fondamentaux révisés : POO, SQL, API REST, Git
QUESTIONS	
☐	Réponses préparées aux questions RH classiques
☐	Deux ou trois questions à poser au recruteur
LOGISTIQUE	
☐	Itinéraire ou lien de visioconférence testé
☐	Matériel et connexion vérifiés
☐	CV, offre et carnet à portée de main
☐	Tenue prête

5 Modèle de cahier des charges

À remplir avec l'encadrant professionnel puis à faire valider.

Rubrique	À renseigner
Titre du projet	[...]
Entreprise et service	[...]
Contexte et problème	[...]
Objectifs (SMART)	[1] [2] [3]
Parties prenantes	[Commanditaire] [Utilisateurs] [Encadrants]
Périmètre — inclus	[...]
Périmètre — exclu	[...]
Exigences fonctionnelles	EF-01 [...] (priorité) EF-02 [...] (priorité)
Exigences non fonctionnelles	Performance [...] Sécurité [...] Ergonomie [...] Maintenabilité [...]
Contraintes	[technologies, sécurité, délais]
Livrables	[application, code, documentation, rapport, présentation]
Critères d'acceptation	[comment chaque exigence sera validée]
Risques et mesures	[risque — probabilité — impact — mesure]
Planning et jalons	[phases et dates clés]
Validation	Nom, rôle, date et signature de l'encadrant : [...]

6 Modèle de planning PFE

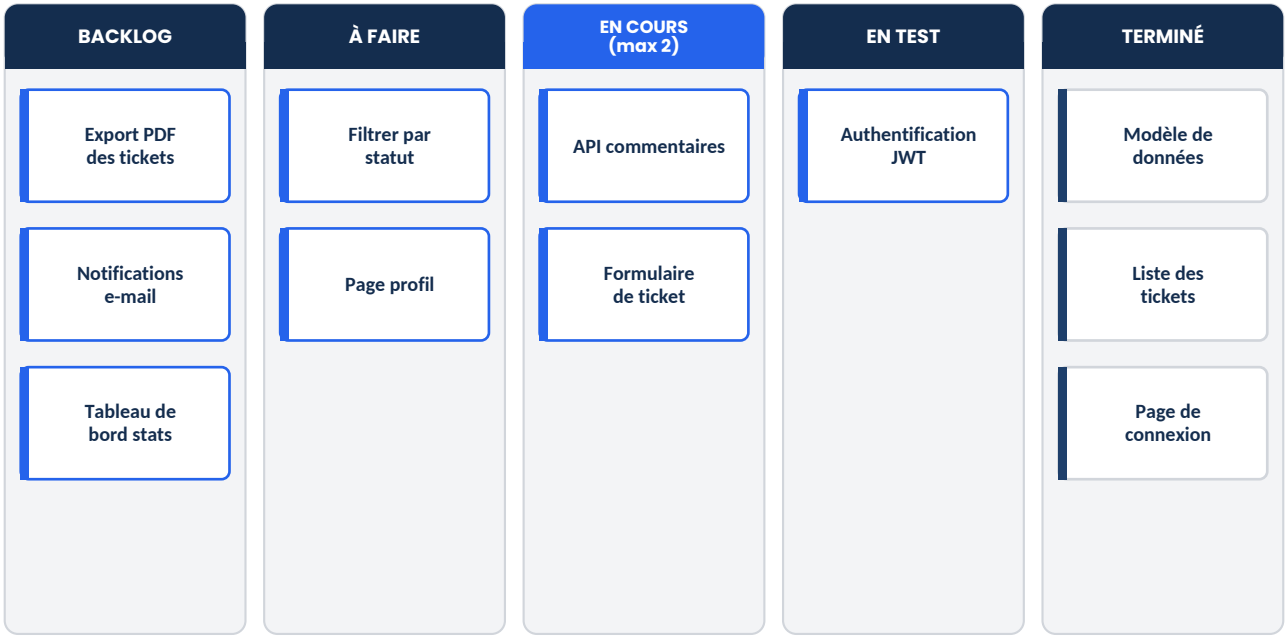
Un tableau simple à compléter avec vos dates réelles.

Phase	Début	Fin	Livrable	Jalon	État

Phases à placer : cadrage · cahier des charges · conception · sprints de développement · tests et recette · déploiement et documentation · rapport · préparation de la soutenance.

7 Exemple de tableau Kanban

Une limite de travail en cours évite de tout démarrer sans rien finir.



Règles : une carte = une tâche livrable en 0,5 à 2 jours ; maximum deux cartes en cours ; une carte « terminée » respecte vos critères d'acceptation (code relu, testé, documenté).

8 Checklist de rédaction du rapport

À parcourir avant chaque envoi à l'encadrant.

RAPPORT	
STRUCTURE	
☐	Gabarit officiel de l'établissement respecté
☐	Page de garde, remerciements, résumé et abstract, tables et listes
☐	Introduction avec problématique et plan
☐	Chaque chapitre introduit et conclu
☐	Conclusion avec bilan, limites et perspectives
QUALITÉ	
☐	Figures et tableaux numérotés, légendés et commentés
☐	Diagrammes cohérents avec l'application réelle
☐	Sources citées et bibliographie complète
☐	Aucun passage copié sans citation
☐	Orthographe et typographie relues
MISE EN FORME	
☐	Styles de titres, numérotation et pagination corrects
☐	Table des matières et listes mises à jour
☐	PDF exporté et relu page par page

9 Checklist de préparation de la soutenance

À valider la veille.

SOUTENANCE	
CONTENU	
☐	Structure de 12 à 15 diapositives, une idée par diapositive
☐	Diagramme d'architecture lisible
☐	Réponses préparées aux 30 questions (au moins 15 rédigées)
DÉMONSTRATION	
☐	Scénario écrit, données de test prêtes
☐	Plan B (vidéo ou captures) enregistré
☐	Application testée sur le matériel de la salle ou le vôtre
JOUR J	
☐	Présentation en PDF et en format source sur deux supports
☐	Chronométrage validé en conditions réelles
☐	Pièces d'identité, convention, rapport imprimé si demandé
☐	Tenue, eau, arrivée en avance

10 Planning des dernières semaines avant la soutenance

Adaptez-le au calendrier officiel de votre établissement, qui prime toujours.

Échéance	Objectifs	Fait
S-6	Fonctionnalités principales terminées ; gel du périmètre ; plan détaillé du rapport validé.	
S-5	Tests et corrections ; déploiement ou environnement de démonstration prêt ; chapitres de conception et réalisation rédigés.	
S-4	Première version complète du rapport envoyée à l'encadrant ; documentation technique terminée.	
S-3	Corrections du rapport ; relecture orthographique ; structure de la présentation validée.	
S-2	Dépôt du rapport selon le calendrier officiel ; démonstration enregistrée en plan B ; première répétition.	
S-1	Répétitions chronométrées devant un public ; préparation des questions du jury ; vérification logistique.	
Veille	Dernière répétition courte ; matériel prêt ; sommeil.	
Jour J	Arriver en avance, tester le matériel, respirer, présenter.	

Pour aller plus loin

Les ressources suivantes sont des **sources officielles** (documentation, standards, guides). Les contenus évoluent : vérifiez toujours la version en vigueur.

Sujet	Ressource	Adresse
Web (HTML, CSS, JavaScript, HTTP)	MDN Web Docs	developer.mozilla.org
React	Documentation officielle	react.dev
Angular	Documentation officielle	angular.dev
Node.js et Express	Documentation officielle	nodejs.org · expressjs.com
Spring Boot	Documentation officielle	spring.io
MySQL	Documentation officielle	dev.mysql.com/doc
MongoDB	Documentation officielle	mongodb.com/docs
Git	Livre Pro Git et documentation	git-scm.com
Commits	Conventional Commits	conventionalcommits.org
UML	Spécification de l'OMG	uml.org
API	OpenAPI Initiative	openapis.org
Sécurité web	OWASP	owasp.org
Agilité	Scrum Guide	scrumguides.org
Docker	Documentation officielle	docs.docker.com



À RETENIR

Un dernier mot

Un PFE réussi n'est pas celui qui utilise la technologie la plus récente : c'est celui qui résout un vrai problème, avec méthode, honnêteté et rigueur, et qui sait l'expliquer.

Prenez votre parcours au sérieux, avancez régulièrement, demandez de l'aide à temps. Bonne chance !

Mahdi Cherif — M.D.S — Created by Mahdi Cherif
Tous droits réservés — M.D.S — Mahdi Cherif

M.D.S

CREATED BY MAHDI CHERIF

Réussir son PFE — de la recherche de stage à la soutenance.

Tous droits réservés — M.D.S — Mahdi Cherif